

Data-Driven Structured Policy Iteration for Homogeneous Distributed Systems

Siavash Alemzadeh¹, Shahriar Talebi², *Member, IEEE*, and Mehran Mesbahi³, *Fellow, IEEE*

Abstract—Control of networked systems, comprised of interacting agents, is often achieved through modeling the underlying interactions. Constructing accurate models of such interactions—in the meantime—can become prohibitive in applications. Data-driven control methods avoid such complications by directly synthesizing a controller from the observed data. In this article, we propose an algorithm referred to as data-driven structured policy iteration (D2SPI), for synthesizing an efficient feedback mechanism that respects the sparsity pattern induced by the underlying interaction network. In particular, our algorithm uses temporary “auxiliary” communication links in order to enable the required information exchange on a (smaller) subnetwork during the “learning phase”—links that will be removed subsequently for the final distributed feedback synthesis. We then proceed to show that the learned policy results in a stabilizing structured policy for the entire network. Our analysis is then followed by showing the stability and convergence of the proposed distributed policies throughout the learning phase, exploiting a construct referred to as the “Patterned monoid.” The performance of data-driven structured policy iteration (D2SPI) is then demonstrated using representative simulation scenarios.

Index Terms—Data-driven policy iteration, networked control systems, Patterned monoids, structured control.

Manuscript received 15 October 2023; revised 19 January 2024; accepted 10 February 2024. Date of publication 14 February 2024; date of current version 29 August 2024. This work was supported in part by the National Science Foundation under Grant SES-1541025 and Grant ECCS-2149470 and in part by the Air Force Office of Scientific Research under Grant FA9550-20-1-0053. An earlier version of this paper was presented at the 2021 IEEE Conference on Decision and Control [DOI: 10.1109/CDC45484.2021.9683331]. In comparison, this article expands on connections to broader related works, provides proofs of all results, and includes the suboptimality guarantees. Extended simulation results have also been included in this version of the manuscript. Recommended by Associate Editor C. Gao. (Siavash Alemzadeh and Shahriar Talebi contributed equally to this work.) (Corresponding author: Shahriar Talebi.)

Siavash Alemzadeh was with the William E. Boeing Department of Aeronautics and Astronautics, University of Washington, Seattle, WA 98195 USA. He is now with Microsoft Corporation, Redmond, WA 98052 USA (e-mail: alemz@uw.edu).

Shahriar Talebi was with the William E. Boeing Department of Aeronautics and Astronautics, University of Washington, Seattle, WA 98195 USA. He is now with the John A. Paulson School of Engineering and Applied Sciences, Harvard University, Cambridge, MA 02138 USA (e-mail: talebi@seas.harvard.edu).

Mehran Mesbahi is with the William E. Boeing Department of Aeronautics and Astronautics, University of Washington, Seattle, WA 98195 USA (e-mail: mesbahi@uw.edu).

Digital Object Identifier 10.1109/TAC.2024.3366038

I. INTRODUCTION

IN RECENT years, there has been a renewed interest in the distributed control of large-scale systems. The unprecedented interdependence and size of the data generated by such systems have necessitated a distributed approach to policy computation in order to influence or direct their behavior and performance. In these scenarios, collective actions are often synthesized via local decisions, informed by a *structured* information exchange mechanism. An important roadblock for centralized control design methods is, thereby, their scalability and shortcomings in utilizing the underlying structure of large-scale interconnected systems.¹

Structured control synthesis in the meantime is generally an NP-hard constrained optimization problem [4]. Hence, distributed control design for large-scale systems has often been pursued not necessarily to characterize optimal policies per se but to devise efficient (possibly suboptimal) control mechanisms that exploit the inherent system structure. In parallel, recent advances in measurement technologies have made available an unprecedented amount of data, motivating how offline and online data processing can be leveraged for data-driven decision making on high-dimensional complex systems.

In this work, we propose the linear-quadratic regulator (LQR)-based algorithm, coined data-driven structured policy iteration (D2SPI), to iteratively learn stabilizing controllers for unknown but identical linear dynamical systems that are connected via a network induced by the coupling in their performance. The setup is a particular realization of cooperative game-theoretic decision making (see remarks under footnote 2). This class of synthesis problems is motivated by applications such as formation flight [5] and distributed camera systems [6], where the dynamics of the network nodes (agents) cannot be precisely parameterized. D2SPI is built upon a data-driven learning phase on a subgraph in a large network. This subgraph includes the agent with maximum degree in the network and requires enabling *auxiliary* links within this subgraph in order to iteratively learn a stabilizing structured controller (optimal for the subgraph) for the entire network (see Fig. 1). This “extension” synthesis procedure utilizes a symmetry property of the networked systems, which we refer to as the *Patterned monoid* (see Section II-A).

The rest of this article is organized as follows. In Section II, we introduce the problem setup and motivation behind our

¹ $\mathcal{O}(n^3)$ complexity of solving the *algebraic Riccati equation* [2] and scalability issues of *model-predictive control* [3] are among such examples.

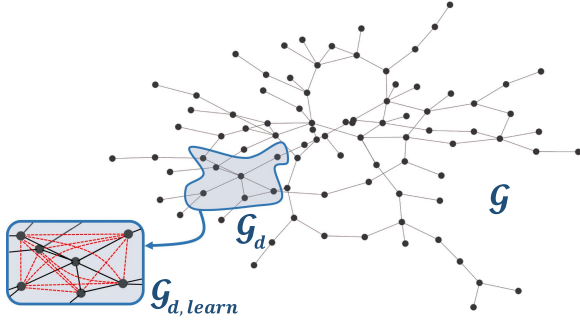


Fig. 1. Addition of auxiliary links (dashed red) to the subgraph $\mathcal{G}_d$ during the policy learning phase. The size of the subgraph depends on the maximum degree of the original graph $\mathcal{G}$.

work and provide an overview of the related literature (see Section II-B). In Section III, we present and analyze the D2SPI algorithm, followed by the theoretical analysis in Section IV. Illustrative examples are provided in Section V. Finally, Section VI concludes this article.

Notation: The operator $\text{diag}(\cdot)$ makes a square diagonal matrix out of the elements of its argument; $\text{vech}(\cdot)$, on the other hand, takes a square matrix and stacks the lower left triangular half (including the diagonal) into a single vector. We use $N \succ 0$ ($\succeq 0$) to declare N as a positive(semi)definite matrix. The i th eigenvalue and spectral radius of M are denoted by $\lambda_i(M)$ and $\rho(M)$; M is called Schur stable when $\rho(M) < 1$. We say that an n -dimensional linear system parameterized by the pair (A, B) is *controllable* if the controllability matrix $\mathcal{C} = [B \ AB \ \dots \ A^{n-1}B]$ has a full rank. We denote the *Kronecker product* of two matrices by $\otimes$. For a block matrix $\mathbf{F}$, by $[\mathbf{F}]_{rk}$, we imply the r th row and k th column “block” component with appropriate dimensions. An (undirected) graph is characterized by $\mathcal{G} = (\mathcal{V}_{\mathcal{G}}, \mathcal{E}_{\mathcal{G}})$, where $\mathcal{V}_{\mathcal{G}}$ is the set of nodes and $\mathcal{E}_{\mathcal{G}} \subseteq \mathcal{V}_{\mathcal{G}} \times \mathcal{V}_{\mathcal{G}}$ denotes the set of edges. An edge exists from node i to j if (the unordered pair) $(i, j) \in \mathcal{E}_{\mathcal{G}}$; this is also specified by writing $j \in \mathcal{N}_i$, where $\mathcal{N}_i$ is the set of neighbors of node i . We designate the maximum degree of $\mathcal{G}$ by $d_{\max}(\mathcal{G})$. Finally, the graph $\mathcal{G}$ can be represented using matrices such as the Laplacian $\mathcal{L}_{\mathcal{G}}$ or the adjacency $\mathcal{A}_{\mathcal{G}}$. To distinguish between system dynamics quantities related to the entire graph and a subgraph, we utilize hat and tilde notation, respectively. A *semigroup* is a set and a binary operator in which the multiplication operation is associative (but its elements need not have inverses). A *monoid* is a semigroup with an identity element. A *group* is a monoid, each of whose elements is invertible. We denote the set of symmetric $n \times n$ real matrices by $\mathbb{S}^n$, and the set invertible ones by $\text{GL}(n, \mathbb{R})$ —which is a group under matrix multiplication also known as the general linear group.

II. PROBLEM SETUP

Consider a network of identical agents with interdependencies induced by a network-level objective. In particular, we assume that the system contains N agents forming a graph $\mathcal{G} = (\mathcal{V}_{\mathcal{G}}, \mathcal{E}_{\mathcal{G}})$, where each node of the graph in $\mathcal{V}_{\mathcal{G}}$ represents a linear discrete-time system

$$\mathbf{x}_{i,t+1} = A\mathbf{x}_{i,t} + B\mathbf{u}_{i,t}, \quad i = 1, 2, \dots, N \quad (1)$$

with $\mathbf{x}_{i,t} \in \mathbb{R}^n$ and $\mathbf{u}_{i,t} \in \mathbb{R}^m$ denoting the state and input of agent i at time step t , respectively. The unknown system matrices $A \in \mathbb{R}^{n \times n}$ and $B \in \mathbb{R}^{n \times m}$ are assumed to form a controllable pair. The network dynamics can compactly be represented as

$$\hat{\mathbf{x}}_{t+1} = \hat{\mathbf{A}}\hat{\mathbf{x}}_t + \hat{\mathbf{B}}\hat{\mathbf{u}}_t \quad (2)$$

where $\hat{\mathbf{x}}_t \in \mathbb{R}^{Nn}$ and $\hat{\mathbf{u}}_t \in \mathbb{R}^{Nm}$ are comprised of the states and inputs of the entire network, $\hat{\mathbf{x}}_t = [\mathbf{x}_{1,t}^\top \dots \mathbf{x}_{N,t}^\top]^\top$, and $\hat{\mathbf{u}}_t = [\mathbf{u}_{1,t}^\top \dots \mathbf{u}_{N,t}^\top]^\top$, with $\hat{\mathbf{A}} \in \mathbb{R}^{Nn \times Nn}$ and $\hat{\mathbf{B}} \in \mathbb{R}^{Nn \times Nm}$ in block diagonal forms $\hat{\mathbf{A}} = I_N \otimes A$ and $\hat{\mathbf{B}} = I_N \otimes B$. The agents’ interconnections are represented by edges $\mathcal{E}_{\mathcal{G}} \subseteq \mathcal{V}_{\mathcal{G}} \times \mathcal{V}_{\mathcal{G}}$ that can facilitate a distributed feedback design. We do not assume that $\mathcal{G}$ is necessarily connected; the motivation for this becomes apparent subsequently. Let $\mathcal{N}_i$ denote the set of neighbors of node i in $\mathcal{G}$ (excluding itself). Then, based on the underlying communication graph and for any choice of positive integers a and b , we define a linear subspace of real-valued $aN \times bN$ matrices as

$$\mathcal{U}_{a,b}^N(\mathcal{G}) := \{\mathbf{M} \in \mathbb{R}^{aN \times bN} \mid [\mathbf{M}]_{ij} = \mathbf{0} \text{ if } j \notin \mathcal{N}_i \cup \{i\},$$

$$[\mathbf{M}]_{ij} \in \mathbb{R}^{a \times b}, \ i, j = 1, \dots, N\}.$$

Without having access to the system parameters A and B , we are interested in designing linear feedback gains, consistent with the desired sparsity pattern induced by the network, using data generated by (2). More precisely, given an initial condition $\hat{\mathbf{x}}_1$, the distributed (structured) optimal control problem assumes the form

$$\begin{aligned} \min_{\hat{\mathbf{K}}} \quad & \sum_{t=1}^{\infty} \hat{\mathbf{x}}_t^\top \hat{\mathbf{Q}} \hat{\mathbf{x}}_t + \hat{\mathbf{u}}_t^\top \hat{\mathbf{R}} \hat{\mathbf{u}}_t \\ \text{s.t.} \quad & (2), \quad \hat{\mathbf{u}}_t = \hat{\mathbf{K}} \hat{\mathbf{x}}_t, \quad \hat{\mathbf{K}} \in \mathcal{U}_{m,n}^N(\mathcal{G}) \end{aligned} \quad (3)$$

where $\hat{\mathbf{K}}$ stabilizes the pair $(\hat{\mathbf{A}}, \hat{\mathbf{B}})$ (i.e., $\rho(\hat{\mathbf{A}} + \hat{\mathbf{B}}\hat{\mathbf{K}}) < 1$), $\hat{\mathbf{R}} = I_N \otimes R$, and $\hat{\mathbf{Q}} = I_N \otimes Q_1 + \mathcal{L}_{\mathcal{G}} \otimes Q_2$ for some given cost matrices $Q_1 \succ 0$, $Q_2 \succeq 0$, and $R \succ 0$. Note that $\hat{\mathbf{Q}} \in \mathcal{U}_{n,n}^N(\mathcal{G})$ is positive definite. Such interdependence induced through the cost has been considered in a graph-based distributed control framework; see, for instance, [7], [8], [9], and [10]. In a nutshell, the first term in $\hat{\mathbf{Q}}$ encodes the cost pertinent to state regulation for each agent, while the second term captures the “disagreement” cost between the neighbors.² In this work, we propose

²One instance of such an interactive cost among agents appears in the cooperative game setup, where agent i aims to solve the minimization problem

$$\begin{aligned} \min_{(\mathbf{u}_{i,t})_{t=0}^{\infty} \in \ell_2} \quad & \mathbf{J}_i(\hat{\mathbf{x}}_1, \hat{\mathbf{u}}_t) = N \sum_{t=0}^{\infty} \left(\mathbf{x}_{i,t}^\top Q_1 \mathbf{x}_{i,t} + \mathbf{u}_{i,t}^\top R \mathbf{u}_{i,t} \right. \\ & \left. + \sum_{j \in \mathcal{N}_i} (\mathbf{x}_{j,t} - \mathbf{x}_{i,t})^\top Q_2 (\mathbf{x}_{j,t} - \mathbf{x}_{i,t}) \right). \end{aligned}$$

Then, it is well known that the set of Pareto front solution of this game can be obtained by minimizing the parametric cost function

$$\min_{(\mathbf{u}_{1,t})_{t=0}^{\infty}, \dots, (\mathbf{u}_{N,t})_{t=0}^{\infty} \in \ell_2} \sum_{i=1}^N \alpha_i \mathbf{J}_i(\hat{\mathbf{x}}_1, \hat{\mathbf{u}}_t)$$

parameterized by $\alpha_1, \dots, \alpha_N$, where $\alpha_i \in [0, 1]$ and $\sum_i \alpha_i = 1$ (see, e.g., [11]). Therefore, a cost such as in (3) can be viewed as a special case of the fair Pareto optimal solution with the choice of $\alpha_i = 1/N$ for all i .

a data-guided suboptimal solution for (3), not relying on the knowledge of system parameters $\hat{\mathbf{A}}$ and $\hat{\mathbf{B}}$. Instead, our approach relies on the system's input-state time series for synthesizing distributed feedback control on $\mathcal{G}$. A summary of challenges in analyzing this problem is listed as follows.

- 1) The constrained optimization problem in (3) is, in general, NP-hard [4], [12]. In particular, the problem of stabilization by decentralized static state feedback is NP-hard if one imposes a bound on the norm of the controller [12]. Even though the result is not shown for the case with no bound on the controller parameters, the general problem is commonly believed to be a computationally hard problem. Note that stabilization is a necessity for the feasibility of the optimization problem in (3) for arbitrary initial state $\mathbf{x}_0$ whenever $Q_1 \succ 0$. This is simply because the cost is unbounded for an unstable policy. Nonetheless, based on the complete knowledge of system parameters, this problem has been investigated under a variety of assumptions [7], [13], [14], [15] or approached directly with the aid of projected gradient-based policy updates [16], [17].
- 2) In the meantime, policies obtained via data-driven approaches do not necessarily respect the hard constraints on $\hat{\mathbf{K}} \in \mathcal{U}_{m,n}^N(\mathcal{G})$ posed in (3). In particular, we point out that “projection” onto the intersection of the constraint imposed by the network and stabilizing controllers is not straightforward due to the intricate geometry of the set of stabilizing controllers [18].
- 3) Another key challenge in adopting data-driven methods for the entire network is rooted in the “curse of dimensionality” inherent in the analysis of large-scale systems. In fact, even collecting data from the entire network can be prohibitive.
- 4) Finally, it is often impossible in applications to pause the operation of the network for data collection or decision-making purposes.³

A. Structures in the Problem and Our Approach

In this work, the sparsity requirement $\hat{\mathbf{K}} \in \mathcal{U}_{m,n}^N(\mathcal{G})$ is considered as a *hard constraint* for control synthesis, and as such, the corresponding optimization is challenging in general. Hence, we shift our attention from the optimal solution of (3) toward a “reasonable” suboptimal stabilizing distributed controller with a scalable computational cost. We aim to exploit the problem structure that is incurred due to the homogeneity of system dynamics and the pattern in performance index respecting the underlying graph topology—see the definition of $\hat{\mathbf{Q}}$ depending on the graph Laplacian $\mathcal{L}_{\mathcal{G}}$ in (3). In addition, as system parameters in (2) are unknown, we adapt a Q -learning procedure to our setup that provably converges to a distributed policy with suboptimality guarantees. In the *absence* of the sparsity constraint $\hat{\mathbf{K}} \in \mathcal{U}_{m,n}^N(\mathcal{G})$, a learning approach for solving the optimal control problem (3) is the well-known Q -learning procedure that was first introduced by Bradtke et al. [19], [20].

³For example, consider an operational large-scale group of homogeneous aerial vehicles that need to coordinate their relative states (in addition to their respective state regulation) over a network induced by their proximity [5].

Adopting this approach for (3) would require utilizing the input-state data trajectories of the entire networked systems in (2) to implement a quasi-Newton method for iteratively updating $\hat{\mathbf{K}}$ introduced by Hwer [21]. If the system is controllable, Hwer's algorithm converges to the globally optimal solution with a quadratic rate, and so does the Bradtke's algorithm if, in addition, the data trajectories are “informative” enough—this is usually satisfied by a sufficient condition on the input signal referred to as “persistence of excitation.” The main issue with this approach is the fact that the policy obtained in this way will not respect the hard constraint of $\hat{\mathbf{K}} \in \mathcal{U}_{m,n}^N(\mathcal{G})$ as was posed in optimization (3). This issue is particularly critical when a projection of the iterated policy on the set $\mathcal{U}_{m,n}^N(\mathcal{G})$ is not practical or even fails to be stabilizing. In addition, collecting data trajectory from entire networked system in (2) can be expensive.

Inspired by the Q -learning approach, we propose a model-free structured policy iteration scheme for the synthesis problem (3) with iterative stability, convergence, and reasonable performance guarantees. While the detailed algorithm is presented subsequently in Section III, in what follows, we summarize the key steps of our design procedure for distributed data-driven policy iteration.

1) Inherent to our distributed learning algorithm is a synthesis subproblem whose dimension is related only to the maximum degree in an underlying graph rather than the dimension of the original network. In particular, we will reason that for the learning phase, our method only requires data collection from a (specific) smaller subnetwork $\mathcal{G}_d \subseteq \mathcal{G}$ with size $d = d_{\max}(\mathcal{G}) + 1$. This subgraph is substantially smaller than the original graph whenever $d_{\max}(\mathcal{G})$ is significantly smaller than N , reflecting the empirical feature of many real-world networks. In the meantime, our approach requires adding temporary communication links to the subgraph $\mathcal{G}_d$ to make it a clique during the learning phase—we will discuss why this learning phase clique is required in Step 3 below—see Fig. 1. The additional links are subsequently removed for the final data-driven feedback design. In robotic applications, such a clique can be established by temporary increasing the transmission range for neighboring nodes of the agent with maximum degree. Since longer range information transfer is generally costly, power levels for these agents are “dialed back” to their original settings following the completion of the learning phase.

2) Then, we adapt the Q -learning technique in order to learn a specific optimal policy for this subproblem using data only from the systems in the subgraph $\mathcal{G}_d$. In particular, the proposed algorithm learns an optimal policy for the subgraph with a (off-)diagonal pattern consisting of two distinct system-level policies K^* and L^* , representing the “individual” and “cooperative” components, respectively. Specifically, for any integer $r \geq 2$, we can define a linear subspace of $\mathbb{R}^{rn \times rn}$ as

$$\begin{aligned} \mathcal{L}(r \times n, \mathbb{R}) &:= \{ \mathbf{N}_r \in \mathbb{R}^{rn \times rn} \mid \mathbf{N}_r \\ &= \mathbf{I}_r \otimes (\mathbf{A} - \mathbf{B}) + \mathbf{1}_r \mathbf{1}_r^T \otimes \mathbf{B}, \text{ for some } \mathbf{A}, \mathbf{B} \} \end{aligned}$$

which is also closed under matrix multiplication. Then, the policy learned for the subgraph $\mathcal{G}_d$ takes the form $\hat{\mathbf{K}}^* = \mathbf{I}_d \otimes$

$(K^* - L^*) + 11^\top \otimes L^*$, which also implies that the closed-loop system $\tilde{\mathbf{A}}_{\tilde{\mathbf{K}}^*} := \tilde{\mathbf{A}} + \tilde{\mathbf{B}}\tilde{\mathbf{K}}^*$ lies in $L(d \times n, \mathbb{R})$. This is particularly useful from a design perspective as it becomes clear in the next step.

3) Next, we define the *Patterned monoid* as

$$\text{PM}(r \times n, \mathbb{R}) := \{ \mathbf{N}_r \in L(r \times n, \mathbb{R}) \cap \text{GL}(rn, \mathbb{R}) \mid \\ \text{for some } A \in \text{GL}(n, \mathbb{R}) \cap \mathbb{S}^n, B \in \mathbb{S}^n \}.$$

Note that the Patterned monoid $\text{PM}(r \times n, \mathbb{R})$ is indeed a submonoid of $\text{GL}(rn, \mathbb{R})$ —following by Lemma 2—i.e., it is closed under matrix multiplications and contains the identity matrix.⁴ The next observation underscores the relevance of Patterned monoids in system analysis.

Proposition 1: For a Schur stable matrix $A \in L(r \times n, \mathbb{R})$ and $0 \prec Q \in \mathbb{S}^{rn}$, let P denote the unique solution to the corresponding discrete-time Lyapunov equation, i.e., $P = A^\top P A + Q$. Then, $P \in \text{PM}(r \times n, \mathbb{R})$ if and only if $Q \in \text{PM}(r \times n, \mathbb{R})$.

The invariance of the Lyapunov equation under the action of Patterned monoid has important implications for our data-driven synthesis for large-scale networks (see Proposition 2). Another key ingredient of our approach, motivated by utilizing the Patterned monoid in the context of Q -learning, involves allowing temporary communication links on the subgraph $\mathcal{G}_d$ in order to make it a clique only *during the learning phase*—links that are subsequently removed. It then follows that the state cost parameter $\tilde{\mathbf{Q}}$ for the subgraph $\mathcal{G}_d$ lies in $\text{PM}(d \times n, \mathbb{R})$. Then, as the obtained closed-loop system $\tilde{\mathbf{A}}_{\tilde{\mathbf{K}}}$ at each iteration of the learning phase lies in $L(d \times n, \mathbb{R})$, Proposition 1 would imply that the associated cost matrix $\tilde{\mathbf{P}}_{\tilde{\mathbf{K}}}$ must lie in $\text{PM}(d \times n, \mathbb{R})$ with two components $P_1 \in \text{GL}(n, \mathbb{R}) \cap \mathbb{S}^n$ and $P_2 \in \mathbb{S}^n$.

4) Next, an iterative Q -learning procedure is designed based on both the patterned structure of the policy with system-level components K and L , and the cost matrix with P_1 and P_2 components. This procedure provably converges to an optimal policy for the subgraph $\mathcal{G}_d$ (see Theorem 2).

5) We note that terminating the operation of the entire networked system for the purpose of data collection/learning is often infeasible in real-world applications. For example, disrupting the operation of power generators and loads for improving their respective network-level performance is highly undesirable. Therefore, in addition to learning the optimal policy for the subgraph, in this work, we aim to simultaneously devise and update a policy for the rest of the network. In this direction, our algorithm iteratively learns a “stability margin” τ according to the learned components of the policy at each iteration (see Proposition 3). This feature, together with homogeneity of the network, facilitates extending the policy synthesis procedure to a stabilizing policy for the entire network by utilizing the individual and cooperative components learned from the subgraph $\mathcal{G}_d$ (see Theorem 1).

⁴The proposed Patterned monoid and characterizing its interplay with the Lyapunov equation is considered as a key contribution in our approach. Note that, with reference to [1], $\text{PM}(r \times n, \mathbb{R})$ requires further qualifications to be a linear subgroup of $\text{GL}(rn, \mathbb{R})$. The monoid characterization is sufficient for our purposes, and further extensions are deferred to our future work.

6) After the Q -learning procedure has converged, the learning phase is terminated. The framework now allows removing the temporary links added to $\mathcal{G}_d$ during the “clique subgraph learning phase,” and the stability of the final learned policy will be guaranteed (see Corollary 1).

7) Finally, note that the learned policy for the entire graph will be, in general, a suboptimal solution to the optimization in (3). Yet, we provide guarantees on its suboptimality gap in Theorem 3 and illustrate its numerical performance in Section V.

The distributed control underpinning of the method proposed in this work follows its model-based analogues studied in [7] and [9]. In this work, our contribution is to build on these approaches and propose a model-free structured policy iteration algorithm, which is not only computationally efficient but also practical for operational large-scaled networked systems.

B. Related Literature

Distributed control is a well-established area of research in systems theory. The roots of the field trace back to the socioeconomics literature in 1970s [22] and early works in the control literature followed suite later during that decade [23]. The main motivation for these works was lack of scalability in centralized planning and control, due to information or computational limitations [24], [25]. Fast forward a few decades, sufficient graph-theoretic conditions were provided for the stability of formations comprised of identical vehicles [26] and, along the same lines, graph-based distributed controller synthesis was further examined independently in works such as [7], [10], [16], and [27]. The topic was also studied from the perspective of spatial invariance [15], [28] and a compositional layered design [29], [30]. Moreover, from an agent-level perspective, the problem has been tackled for both homogeneous systems [7], [9], [10] and, more recently, heterogeneous ones [31].

Having access to the underlying system model is a common assumption in the literature on distributed control, where the goal is to find a distributed feedback mechanism that conforms to an underlying network topology. However, deriving dynamic models from first principles could be restrictive for large-scale systems and complex mission scenarios [32]. Such restrictions also hold for parametric perturbations that occur due to inefficient modeling or other unknown design factors. For instance, even the LQR solution with its strong input robustness properties may have small stability margins for general parameter perturbations [33]. Robust synthesis approaches could alleviate this issue when the perturbations follow specific models in both centralized [34] and distributed [35] cases. However, if the original estimates of system parameters are inaccurate or the perturbations violate the presumed model, then both stability and optimality of the proposed feedback mechanisms can be compromised. Data-driven control, on the other hand, circumvents such drawbacks and utilize the available data generated by the system when its model is unavailable. This point of view has historically been examined in the context of adaptive control and

system identification [36], particularly, when asymptotic properties of the synthesized system are of interest. For more recent works that have adopted a nonasymptotic outlook on data-driven control, we mention [37], [38], and [39] that used batched data for synthesis, as well as online iterative procedures [40], [41]. Furthermore, in regard to the adaptive nature of such algorithms, there is a close connection between online data-driven control and reinforcement learning [19], [42]. In these latter works, policy iteration has been extended to approximate LQR by avoiding the direct solution of algebraic Riccati equation (ARE); yet, majority of these works do not have favorable scaling properties.

Control and estimation for large-scale systems offers its unique set of challenges due to higher levels of uncertainty, scalability issues, and modeling errors. Nevertheless, model-free synthesis for large-scale systems, as a discipline, is still in its infancy. From a control theoretic perspective, the work [43] addresses some of the aforementioned issues using ideas from mean-field multiagent systems and with the key assumption of partial exchangeability. The work [44], on the other hand, provides a decentralized LQR algorithm based on network consensus that has low complexity, but potentially a high cost of implementation. Finally, semidefinite programming (SDP) projection-based analysis has been examined in [45], where each agent will have a sublinear regret as compared with the best fixed controller in hindsight. The problem has also been considered from a game-theoretic standpoint [46], [47], [48], where agents can have conflicting objectives.

In the following section, we propose our algorithm that iteratively learns necessary control components from a subnetwork that would be used to design a distributed controller for the entire network. We also show that depending on the structure of the problem, not only would our scheme be computationally efficient but also more applicable when model-based control in high dimensions is (practically) infeasible. The structure of our distributed control design is inspired by Borrelli and Keviczky [7], which was subsequently extended to discrete time in [9].

III. MAIN ALGORITHM

In this section, we present and discuss the main algorithm of this article, namely, D2SPI. Given the underlying communication graph $\mathcal{G}$, the networked system is considered as a black box, whereas the designer is capable of injecting input signals to the system and observe the corresponding states. The goal of D2SPI is then to find a data-guided suboptimal solution for (3) without the knowledge of system parameters $\hat{\mathbf{A}}$ and $\hat{\mathbf{B}}$. To this end, our approach involves considering the synthesis problem on a subgraph $\mathcal{G}_d \subseteq \mathcal{G}$, with the associated time-series data. Before presenting the main algorithm, we formalize two useful notions in order to facilitate the presentation.

Definition 1: Given a subgraph $\mathcal{G}' \subseteq \mathcal{G}$ and a node labeling, let $\text{Policy}(\mathcal{V}_{\mathcal{G}'})$ denote the concatenation of policies of the agents in $\mathcal{V}_{\mathcal{G}'}$, i.e., $\text{Policy}(\mathcal{V}_{\mathcal{G}'}) := [\mathbf{u}_1^\top \ \mathbf{u}_2^\top \ \cdots \ \mathbf{u}_{|\mathcal{V}_{\mathcal{G}'}|}^\top]^\top$, where $\mathbf{u}_i$ is the feedback control policy of agent i in the subgraph $\mathcal{G}'$ as a

mapping from $\{\mathbf{x}_j | j \in \mathcal{N}_i \cup \{i\}\}$ to $\mathbb{R}^m$. Furthermore, we use $\text{Policy}(\mathcal{V}_{\mathcal{G}'})|_t$ to denote the realization of these policies at time t . Similarly, we define $\text{State}(\mathcal{V}_{\mathcal{G}'}) := [\mathbf{x}_1^\top \ \mathbf{x}_2^\top \ \cdots \ \mathbf{x}_{|\mathcal{V}_{\mathcal{G}'}|}^\top]^\top$.

The D2SPI algorithm is introduced in Algorithm 1 with the following standard assumption.

Assumption 1: The initial controller K_1 is stabilizing for the controllable pair (A, B) , and $\mathbf{e}_t$ in Algorithm 2 is such that $\text{Policy}(\mathcal{V}_{\mathcal{G}'})|_t$ remains persistently exciting (PE); more precisely, if we collect the state and input difference signals in the vector form ϕ_t , as defined in Algorithm 2, then the PE condition requires that [20] there exists a positive integer N_0 and positive constants $\epsilon_0 \leq \bar{\epsilon}_0$ such that

$$\epsilon_0 I \leq \frac{1}{N} \sum_{i=1}^N \phi_{t-i} \phi_{t-i}^\top \leq \bar{\epsilon}_0 I \quad \text{for all } t \geq N_0 \text{ and } N \geq N_0.$$

Remark 1: Note that the controllability of the pair (A, B) and the PE condition are sufficient assumptions to guarantee well-posedness of the data-driven control problem that are commonly adopted in literature [49], [50]. This PE condition is an adaptation of “strong persistent excitation” in [49] to the least-squares problem of recovering unknown parameters of the so-called $\mathcal{Q}$ -function. The PE condition can be easily satisfied in our setup by ensuring a rich randomness in the signal $\mathbf{e}_t$, such as a nondegenerate Gaussian distribution. A more modern treatment of PE condition is stated by a rank condition on the input Hankel matrices in the context of Willems’ fundamental lemma [50], [51]. We also refer to [51] for a discussion on how these assumptions can be relaxed. Furthermore, assuming initial stabilizing controller K_1 is also common in the policy iteration literature. For instance, in the case of open-loop stable system A , K_1 is simply chosen to be zero. For an unknown and unstable pair (A, B) , more elaborate online stabilization techniques have been adopted that are out of the scope of this work. Refer to [52] for one such method.

A. Learning Phase

We refer to the main loop of the algorithm in Line 8 as the *learning phase*. During the learning phase, we include temporary “auxiliary” links to $\mathcal{G}_d$ and make the communication graph a clique. We show such distinction by $\mathcal{G}_{d,\text{learn}}$, where $|\mathcal{V}_{\mathcal{G}_d}| = |\mathcal{V}_{\mathcal{G}_{d,\text{learn}}}|$, but $\mathcal{G}_{d,\text{learn}}$ is a clique. Inherent to D2SPI is a policy iteration on $\mathcal{G}_{d,\text{learn}}$ that characterizes components K_k and L_k , intuitively representing “self” and “cooperative” controls at iteration k , respectively. In particular, during the learning phase, we utilize these control components in order to design and update an effective stabilizing controller for the rest of the network $\mathcal{G} \setminus \mathcal{G}_{d,\text{learn}}$.

We do so by ensuring that during the learning phase, information is exchanged unidirectionally from $\mathcal{G}_{d,\text{learn}}$ to the rest of the network; hence, the policy of the agents in $\mathcal{G} \setminus \mathcal{G}_{d,\text{learn}}$ is dependent on those in $\mathcal{G}_{d,\text{learn}}$, and not vice versa. After the learning phase terminates, we remove the temporary links added during the learning phase (reinitialize to the original network topology) and synthesize a suboptimal stabilizing control for the entire network. In the learning phase of D2SPI, we use a

Algorithm 1: Data-Driven Structured Policy Iteration.

-
- 1: **Initialization** ($t \leftarrow 1, k \leftarrow 1$)
 - 2: Choose $\mathcal{G}_d \subseteq \mathcal{G}$ with $d = d_{\max}(\mathcal{G}) + 1$
 - 3: Obtain Q_1, Q_2, R , and set $Q_d \leftarrow Q_1 + dQ_2$
 - 4: Get $\tilde{\mathbf{x}}_1 \in \mathbb{R}^{dn}$ and $\tilde{\mathbf{H}}_0 \in \mathbb{R}^{p \times p}$, $p = d(n+m)$
 - 5: Set $\mathcal{P}_0 \leftarrow \beta \mathbf{I}_{p(p+1)/2}$ for large $\beta > 0$ and fix Σ
 - 6: Set K_1 stabilizing (1), $L_1 = \mathbf{0}$, $\Delta K_1 = K_1$
 - 7: Turn on temporary links in $\mathcal{G}_d$ and set $\tau_1 \leftarrow 0$
 - 8: **While** ((K_k, L_k) **has not converged**, **do** (“learning phase”)
 - 9: Set Policy $_k(\mathcal{V}_{\mathcal{G}})$ such that for each $i \in \mathcal{V}_{\mathcal{G}}$,

$$\mathbf{u}_i \leftarrow \Delta K_k \mathbf{x}_i + L_k \sum_{j \in \mathcal{N}_i} \frac{\tau_k}{d-1} \mathbf{x}_j, \quad \text{if } i \in \mathcal{V}_{\mathcal{G} \setminus \mathcal{G}_d}$$

$$\mathbf{u}_i \leftarrow \Delta K_k \mathbf{x}_i + L_k \sum_{j \in \mathcal{V}_{\mathcal{G}_d}} \mathbf{x}_j, \quad \text{if } i \in \mathcal{V}_{\mathcal{G}_d}$$
 - 10: Evaluate $\tilde{\mathbf{H}}_k$ from Algorithm 2
 - 11: $\tilde{\mathbf{H}}_k \leftarrow \text{SPE}(\mathcal{G}, \mathcal{G}_{d,\text{learn}}, \text{Policy}_k(\mathcal{V}_{\mathcal{G}}), \tilde{\mathbf{H}}_{k-1}, \mathcal{P}_0)$
 - 12: Recover X_1, X_2, Y_1, Y_2, Z_1 , and Z_2 from $\tilde{\mathbf{H}}_k$

$$X_1 \leftarrow \tilde{\mathbf{H}}_k[1:n, 1:n]$$

$$Y_1 \leftarrow \tilde{\mathbf{H}}_k[dn+1:dn+m, dn+1:dn+m]$$

$$Z_1 \leftarrow \tilde{\mathbf{H}}_k[dn+1:dn+m, 1:n]$$

$$X_2 \leftarrow \tilde{\mathbf{H}}_k[1:n, n+1:2n]$$

$$Y_2 \leftarrow \tilde{\mathbf{H}}_k[dn+1:dn+m, dn+m+1:dn+2m+1]$$

$$Z_2 \leftarrow \tilde{\mathbf{H}}_k[dn+1:dn+m, n+1:2n]$$

$$\Delta X \leftarrow X_1 - X_2, \quad \Delta Y \leftarrow Y_1 - Y_2, \quad \Delta Z \leftarrow Z_1 - Z_2.$$
 - 12: Update the control components

$$F^{-1} \leftarrow Y_1 - (d-1)Y_2(Y_1 + (d-2)Y_2)^{-1}Y_2$$

$$G \leftarrow (Y_1 + (d-1)Y_2)^{-1}Y_2(Y_1 - Y_2)^{-1}$$

$$K_{k+1} \leftarrow -FZ_1 + (d-1)GZ_2,$$

$$L_{k+1} \leftarrow -FZ_2 + GZ_1 + (d-2)GZ_2,$$

$$\Delta K_{k+1} \leftarrow K_{k+1} - L_{k+1}$$
 - 13: Obtain the stability margin

$$\Xi_{k+1} \leftarrow \Delta X - Q_d + \Delta K_{k+1}^T \Delta Z$$

$$+ \Delta Z^T \Delta K_{k+1} + \Delta K_{k+1}^T (\Delta Y - R) \Delta K_{k+1}$$

$$\gamma_{k+1} \leftarrow \lambda_{\min}(\Delta K_{k+1}^T R \Delta K_{k+1} + Q_d)$$

$$/ \lambda_{\max}(\Xi_{k+1} + L_{k+1}^T (\Delta Y - R) L_{k+1})$$

$$\tau_{k+1} \leftarrow \sqrt{\gamma_{k+1}^2 / (1 + \gamma_{k+1})}$$
 - 14: Go to Line 8 and set $k \leftarrow k + 1$
 - 15: Switch OFF the temporary links and retrieve $\mathcal{G}_d$
 - 16: Set Policy $_k(\mathcal{V}_{\mathcal{G}})$ such that for each $i \in \mathcal{V}_{\mathcal{G}}$,

$$\mathbf{u}_i \leftarrow \Delta K_k \mathbf{x}_i + \frac{\tau_k}{d-1} L_k \sum_{j \in \mathcal{N}_i} \mathbf{x}_j$$
-

recursive least-squares (RLS)-based recursion to estimate the unknown parameters of the Q -function at iteration k , referred to as $\tilde{\mathbf{H}}_k$.

B. Subgraph Policy Evaluation Subroutine

This process is performed in subgraph policy evaluation (SPE) (see Algorithm 2) subroutine by inputting (sub)graph $\mathcal{G}, \mathcal{G}_{d,\text{learn}}$, the mapping policy $(\mathcal{V}_{\mathcal{G}_d})$, and the previous estimate of $\tilde{\mathbf{H}}_{k-1}$. As will be discussed in Appendix A-A, $\tilde{\mathbf{H}}_k$ contains the required information to determine the two control components K_k and L_k from data. We extract this square matrix through a recursive update on the vector θ_{k-1} , derived from half-vectorization of $\tilde{\mathbf{H}}_{k-1}$, solving RLS for the linear equation $\mathcal{R}(\tilde{\mathbf{x}}_t, \tilde{\mathbf{u}}_t) = \zeta_t^T \theta_{k-1}$, where $\mathcal{R}(\tilde{\mathbf{x}}_t, \tilde{\mathbf{u}}_t)$ denotes the local cost and $\zeta_t \in \mathbb{R}^p$ contains the data measurements. We use subscript k for policy update and t for data collection. The adaptive nature of the algorithm involves the exploration signal e_t to be augmented to the policy vector in order to provide the persistence of excitation.

C. Persistence of Excitation and Convergence of SPE

In our setup, e_t is sampled from a normal distribution $e \sim \mathcal{N}(\mathbf{0}, \Sigma)$, where the choice of the variance $\Sigma \succ 0$ is problem specific. In practice, excitation of the input is a subtle task and has been realized in a variety of forms such as random noise [19], sinusoidal signals [53], and exponentially decaying noise [54]. We denote by $\mathcal{P}$ the projection factor that is reset to $\mathcal{P}_0 \succ 0$ for each iteration. Convergence of SPE—guaranteed based on the persistence of excitation condition—is followed by the update of $\tilde{\mathbf{H}}_k$ that encodes the necessary information to obtain K_k and L_k .

D. Learning Control Components Using Patterned Monoid

Learning the control components K_k and L_k is achieved by first recovering the block matrices X_1, X_2, Y_1, Y_2, Z_1 , and Z_2 from $\tilde{\mathbf{H}}_k$ that are further utilized to form intermediate variables F and G . Matrix inversions on line 12 of Algorithm 1 will be justified in Section IV and Lemma 2. Such recovery of meaningful blocks from $\tilde{\mathbf{H}}_k$ is due to the specific structure resulting from adding extra links to $\mathcal{G}_d$ that is captured systematically by the Patterned monoid; this point will be discussed further subsequently in Proposition 2.

E. Learning Gain Margin and the Distributed Feedback Design

Each iteration loop is completed by updating the parameters γ_k and τ_k that prove instrumental in the stability analysis of the proposed distributed controller for the entire network. Finally, with the convergence of D2SPI, $\mathcal{G}_d$ is retrieved by removing the temporary links and the structured policy is extended to the entire graph $\mathcal{G}$ in line 16.

Let us point out a few remarks on the computational complexity of the proposed algorithm. First, note that the inverse operations on line 12 occur on matrices of size $m \times m$ and, hence, computationally inexpensive. Furthermore, the complexity of finding extreme singular values—as on line 13 in Algorithm 1—is known to be $\mathcal{O}(n^2)$ [55]. Hence, the computational complexity of D2SPI is mainly due to the SPE recursion that is equivalent

to the complexity of RLS for the number of unknown system parameters in $\mathcal{G}_d$, i.e., the computational cost is $\mathcal{O}(d^2(n+m)^2)$ [56]. This implies that the computational complexity of the algorithm is *fixed* for any number of agents N , as long as the maximum degree of the graph retains its order.

Remark 2: Adding temporary links within the subgraph $\mathcal{G}_d$ is an effective means of learning optimal K_k and L_k for the subgraph $\mathcal{G}_{d,\text{learn}}$ by utilizing dynamical interdependencies among the agents. Although initializing K_k such that (1) is Schur stable is a standard assumption in data-driven control, obtaining this initial gain for an unknown system is nontrivial. While we invoke this assumption in this work, the interested reader is referred to [40] and [57] for more recent works pertaining to this assumption and related system-theoretic intricacies [51].

IV. CONVERGENCE AND STABILITY ANALYSES

In this section, we provide convergence and stability analyses for the D2SPI algorithm. First, we study the structure and stability margins of each local controller and proceed to establish stability properties of the proposed controller for the entire network throughout the learning process. Finally, we show the convergence of D2SPI to a stabilizing suboptimal distributed controller followed by the derivation of a suboptimality bound characterized by the problem parameters. For clarity, we defer some of the subtleties of the analysis and detailed proofs to Appendix A.

First, let us demonstrate how a specific structure and stability of the controller for the subgraph $\mathcal{G}_{d,\text{learn}}$, when properly initialized, can be preserved throughout the D2SPI algorithm.

Proposition 2: Let $\tilde{\mathbf{K}}_k := \mathbf{I}_d \otimes (K_k - L_k) + \mathbf{1}\mathbf{1}^\top \otimes L_k$ for all $k \geq 1$, with K_k and L_k as in Algorithm 1. Under Assumption 1 and throughout the learning phase (for all $k \geq 1$), $\tilde{\mathbf{K}}_k$ is stabilizing for the system in $\mathcal{G}_{d,\text{learn}}$ and $\text{Policy}_k(\mathcal{V}_{\mathcal{G}_{d,\text{learn}}})|_t = \tilde{\mathbf{K}}_k \text{State}(\mathcal{V}_{\mathcal{G}_{d,\text{learn}}})|_t$, for all t . Furthermore, $\Delta K_k := K_k - L_k$ stabilizes the dynamics of a single agent, i.e., $A + B\Delta K_k$ is Schur stable.

Note that Proposition 2 proves the existence of a stabilizing controller ΔK_k and its corresponding cost-to-go matrix ΔP_k . In the following, our goal is to design a distributed suboptimal controller for the entire networked system on $\mathcal{G}$ based on the components that shape ΔK_k . This extension is built upon the stability margin derived next.

Proposition 3: At each iteration $k \geq 1$, let K_k , L_k , and τ_k be obtained via Algorithm 1. Then, $A + B(K_k - \alpha L_k)$ is Schur stable for all α satisfying $|\alpha - 1| \leq \tau_k$.

The stability margin τ_k in Proposition 3 is upper-bounded by the stability margin of the pair $(A + B(K_k - L_k), B)$. This implies that if the original closed-loop system for an agent does not have a favorable stability margin, then τ_k can be small—reducing the influence of the agent's neighbors on its policy (line 16 of Algorithm 1). Nonetheless, Proposition 3 provides model-free stability gain margins τ_k at each iteration of the algorithm for the dynamics of a single agent in $\mathcal{G}$. In our analysis, we take advantage of these margins to characterize stability guarantees

Algorithm 2: Subgraph Policy Evaluation.

- 1: **Input:** Graph $\mathcal{G}$, subgraph $\mathcal{G}' \subseteq \mathcal{G}$, $\text{Policy}(\mathcal{V}_{\mathcal{G}})$, $\mathbf{H}$, $\mathcal{P}$
 - 2: **Output:** Updated cost matrix $\mathbf{H}^+$ associated with $\mathcal{G}'$
 - 3: **While** $\mathbf{H}$ **has not converged**, **do**
 - 4: Set $\tilde{\mathbf{x}}_t \leftarrow \text{State}(\mathcal{V}_{\mathcal{G}'}|_t$ and $\tilde{\mathbf{u}}_t \leftarrow \text{Policy}(\mathcal{V}_{\mathcal{G}'}|_t$
 - 5: Choose $\mathbf{e}_t \sim \mathcal{N}(\mathbf{0}, \Sigma)$ and update $\text{Policy}(\mathcal{V}_{\mathcal{G}'}|_t$ as $\tilde{\mathbf{u}}_t \leftarrow \tilde{\mathbf{u}}_t + \mathbf{e}_t$ for all $i \in \mathcal{V}_{\mathcal{G}'}$
 - 6: Run $\mathcal{G}$ under policy $\text{Policy}(\mathcal{V}_{\mathcal{G}'})$
 - 7: Collect $\text{State}(\mathcal{V}_{\mathcal{G}'}|_{t+1})$ only from $\mathcal{G}'$ and set $\tilde{\mathbf{x}}_{t+1} \leftarrow \text{State}(\mathcal{V}_{\mathcal{G}'}|_{t+1}$, $\tilde{\mathbf{u}}_{t+1} \leftarrow \text{Policy}(\mathcal{V}_{\mathcal{G}'}|_{t+1}$
 - 8: Set $\phi_t \leftarrow [\tilde{\mathbf{x}}_t^\top \tilde{\mathbf{u}}_t^\top]^\top - [\tilde{\mathbf{x}}_{t+1}^\top \tilde{\mathbf{u}}_{t+1}^\top]^\top$
 - 9: Compute $\zeta_t = \text{vech}(\phi_t \phi_t^\top)$ and $\mathcal{R}(\tilde{\mathbf{x}}_t, \tilde{\mathbf{u}}_t) = \tilde{\mathbf{x}}_t^\top (\mathbf{I} \otimes Q_d - \mathbf{1}\mathbf{1}^\top \otimes Q_2) \tilde{\mathbf{x}}_t + \tilde{\mathbf{u}}_t^\top (\mathbf{I} \otimes R) \tilde{\mathbf{u}}_t$
 - 10: Set $\theta \leftarrow \text{vech}(\mathbf{H})$ and update $\theta \leftarrow \theta + \mathcal{P} \zeta_t (\mathcal{R}(\tilde{\mathbf{x}}_t, \tilde{\mathbf{u}}_t) - \zeta_t^\top \theta) / (1 + \zeta_t^\top \mathcal{P} \zeta_t)$, $\mathcal{P} \leftarrow \mathcal{P} - \mathcal{P} \zeta_t \zeta_t^\top \mathcal{P} / (1 + \zeta_t^\top \mathcal{P} \zeta_t)$
 - 11: Find $\mathbf{H}^+ = \text{vech}^{-1}(\theta)$, update $\mathbf{H} \leftarrow \mathbf{H}^+$ and $t \leftarrow t + 1$
-

for the controller proposed during the learning phase. This is captured in the following result.

Theorem 1: Suppose that K_k , L_k , and τ_k are defined as in Algorithm 1. Then, under Assumption 1, the control policy $\text{Policy}_k(\mathcal{V}_{\mathcal{G}})$ designed during the learning phase (line 9) stabilizes the network $\mathcal{G}$ at each iteration of the learning phase and for any choice of $\mathcal{V}_{\mathcal{G}_d}$.

Theorem 1 establishes that the proposed feedback mechanism stabilizes the entire network, facilitating control of agents *outside* of $\mathcal{G}_d$, during the learning phase. In the meantime, the practicality and suboptimality of the algorithm depend on its convergence addressed next.

Theorem 2: Under Assumption 1 and (long enough) finite termination of Algorithm 2, Algorithm 1 converges, i.e., $K_k \rightarrow K^*$, $L_k \rightarrow L^*$, and $\tau_k \rightarrow \tau^*$ as $k \rightarrow \infty$, where $\tilde{\mathbf{K}}^* = \mathbf{I}_d \otimes (K^* - L^*) + \mathbf{1}\mathbf{1}^\top \otimes L^*$ is the optimal solution to the infinite-horizon state-feedback LQR problem with system parameters $(\tilde{\mathbf{A}}, \tilde{\mathbf{B}}, \tilde{\mathbf{Q}}, \text{ and } \tilde{\mathbf{R}})$ defined as $\tilde{\mathbf{A}} = \mathbf{I}_d \otimes A$, $\tilde{\mathbf{B}} = \mathbf{I}_d \otimes B$, $\tilde{\mathbf{Q}} = \mathbf{I}_d \otimes (Q_1 + dQ_2) - \mathbf{1}\mathbf{1}^\top \otimes Q_2$, and $\tilde{\mathbf{R}} = \mathbf{I}_d \otimes R$.

Finally, we note that as the temporary links introduced during the learning phase are removed, the structure of the agents' interaction is once again the original network $\mathcal{G}$. As such, it is vital to provide stability guarantees after Algorithm 1 terminates and components of the control design have converged. This issue is addressed in the following corollary whose proof is similar to that of Theorem 1 and thus omitted.

Corollary 1: Suppose that K^* , L^* , γ^* , and τ^* are given as in Theorem 2 under a convergent Algorithm 1. Then, $\text{Policy}(\mathcal{V}_{\mathcal{G}})$ (defined on line 16) stabilizes the entire networked system in (2).

We conclude this section by exploring the suboptimality of the proposed policy. Given the problem parameters, let $\tilde{\mathbf{K}}_{\text{struc}}^*$ denote the globally optimal distributed solution for the structured LQR problem in (3) with the associated cost matrix $\tilde{\mathbf{P}}_{\text{struc}}^*$. Given any

other stabilizing structured policy $\hat{\mathbf{K}}$ associated with cost matrix $\hat{\mathbf{P}}$, we define the *optimality gap* as $\text{gap}(\hat{\mathbf{K}}) := \text{tr}[\hat{\mathbf{P}} - \hat{\mathbf{P}}_{\text{struc}}]$. The following theorem provides an upper bound on the optimality gap of structured policy learned by D2SPI based on the problem parameters. In particular, when the system is “contractible,” the derived upper bound depends on the difference of the distributed controller with that of *unstructured* optimal LQR controller.

Theorem 3: Let $\hat{\mathbf{K}}^*$ be the structured policy learned by Algorithm 1 at convergence, corresponding to the cost matrix $\hat{\mathbf{P}}^*$. Moreover, let $\hat{\mathbf{K}}_{\text{lqr}}$ denote the optimal (unstructured) solution to the infinite-horizon state-feedback LQR problem with parameters $(\hat{\mathbf{A}}, \hat{\mathbf{B}}, \hat{\mathbf{Q}}, \text{ and } \hat{\mathbf{R}})$ with the cost matrix $\hat{\mathbf{P}}_{\text{lqr}}$. If $\hat{\mathbf{A}}_{\hat{\mathbf{K}}_{\text{lqr}}} = \hat{\mathbf{A}} + \hat{\mathbf{B}}\hat{\mathbf{K}}_{\text{lqr}}$ is contractible, then $0 \leq \text{gap}(\hat{\mathbf{K}}^*) \leq \text{tr}(\mathbf{M})/[1 - \sigma_{\max}^2(\hat{\mathbf{A}}_{\hat{\mathbf{K}}_{\text{lqr}}})]$, where $\mathbf{M} := (\hat{\mathbf{R}} + \hat{\mathbf{B}}^T \hat{\mathbf{P}}^* \hat{\mathbf{B}})(\hat{\mathbf{K}}^* \hat{\mathbf{K}}^{*T} - \hat{\mathbf{K}}_{\text{lqr}} \hat{\mathbf{K}}_{\text{lqr}}^T) + 2\hat{\mathbf{A}}^T \hat{\mathbf{P}}^* \hat{\mathbf{B}}(\hat{\mathbf{K}}^* - \hat{\mathbf{K}}_{\text{lqr}})$.

Remark 3: First, recall how the converged policy by Theorem 2 is related to the optimal LQR policy on the fully connected subgraph $\mathcal{G}_{d,\text{learn}}$. Second, the optimality gap is characterized by $\text{tr}(\mathbf{M})$, which is essentially proportional to the difference $\hat{\mathbf{K}}^* - \hat{\mathbf{K}}_{\text{lqr}}$, i.e., how close our designed structured policy is to the unstructured LQR that directly depends on the connectivity of the graph topology. Third, the contractibility of the pair (A, B) is more restrictive condition than regularizability of the system [40], a notion that has recently been employed in iterative data-guided control methods [58], [59]. Contractibility also facilitates the validity of assuming access to the initial stabilizing controller.

V. SIMULATION EXAMPLES

In this section, we examine the performance and convergence of D2SPI. In order to assess the suboptimality of the synthesized controllers, we report the trace of cost matrices, $\text{tr}(\hat{\mathbf{P}}_k)$, associated with the proposed distributed controller learned by D2SPI at iteration k . As the optimal distributed design is unknown, we compare these results against the optimal cost for the unconstrained LQR problem, $\text{tr}(\hat{\mathbf{P}}_{\text{LQR}})$, obtained via the solution of the ARE with parameters $(\hat{\mathbf{A}}, \hat{\mathbf{B}}, \hat{\mathbf{Q}}, \text{ and } \hat{\mathbf{R}})$. Note that this is an infeasible solution to the problem in (3); nevertheless, it provides a theoretical lower bound to evaluate the performance of any feasible solution—including the optimal one.⁵

A. Convergence—Randomly Selected Parameters

In the first example, we sample continuous-time system parameters of a single agent (A, B) from a zero-mean normal distribution with unit covariance, such that $A \in \mathbb{R}^{5 \times 5}$ and $B \in \mathbb{R}^{5 \times 3}$. We then consider a path graph of ten agents and demonstrate how Algorithm 1 converges for this network using different instances of the system parameters. The continuous-time system dynamics of a single agent is discretized with a sampling rate of $\Delta T = 0.1$ s. We set $Q_1 = 0.2\mathbf{I}_5$, $Q_2 = \mathbf{I}_5$, and

⁵ All the simulations were run on a 3.2-GHz quad-core Intel Core i5 CPU and in MATLAB. The scripts are publicly available at <https://github.com/shahriarta/D2SPI>.

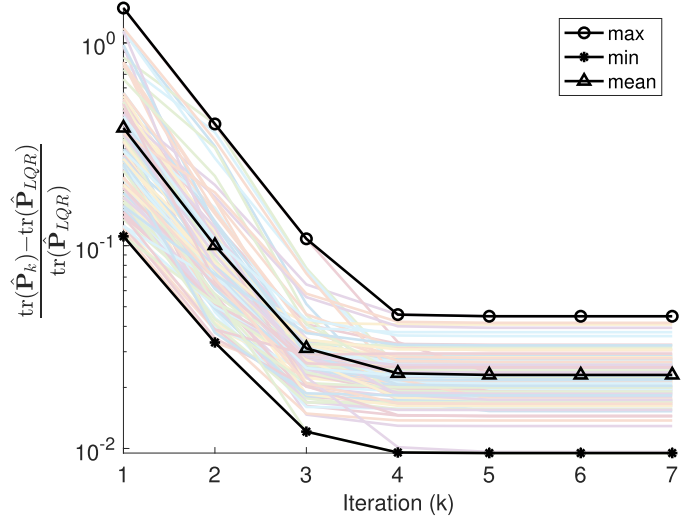


Fig. 2. Convergence of Algorithm 1 for 100 randomly sampled system parameters (A, B) .

$R = \mathbf{I}_3$ in (3). We assume a random exploration signal sampled from a normal distribution $e_t \sim \mathcal{N}(0, \sigma^2)$, where the variance σ^2 is chosen accordingly for different input channels.

Fig. 2 shows the performance of the synthesized controller by illustrating the normalized suboptimality error for the entire network with respect to the (infeasible) LQR controller. This figure depicts the simulation results for 100 random system parameters and shows the progress of the proposed method at each iteration in Algorithm 1. The actual simulations are plotted in faded color, whereas their statistics are plotted in solid black. Note that almost all realizations of the network have converged after five iterations of the learning phase, due to its fast quadratic convergence rate. Assessing the suboptimality of the proposed controller compared against the (infeasible) centralized LQR controller reveals an average improvement by a factor of 200.

B. Network of Homogeneous Plants

In this example, we apply D2SPI to two other simulation scenarios involving homogeneous networks of agents with unknown and unstructured model uncertainties. In particular, we use the dynamics of plants with continuous-time system parameters (A, B) (as reported in [60, Appendix F]), in conjunction with random d -regular graph topologies of different sizes. We then examine the efficacy of D2SPI by illustrating the cost associated with the proposed distributed controller as a function of nodes in the graph. The rest of the problem parameters are chosen identical to the setup in Section V-A. Uncertainty in the model are introduced in this example as follows. Each agent follows an unknown linear time-invariant dynamics similar to (1) with A replaced by $A + \Delta A$, where entries of ΔA are sampled from a normal distribution $\mathcal{N}(0, 0.05)$. Assuming that one has access to a stabilizing controller K_1 for the system with nominal parameters A and B , we set the initial stabilizing controller to be the LQR optimal controller with parameters (A, B, Q_1, R) .

Fig. 3 shows the results of the second simulation example, illustrating how the cost of the proposed controller changes with

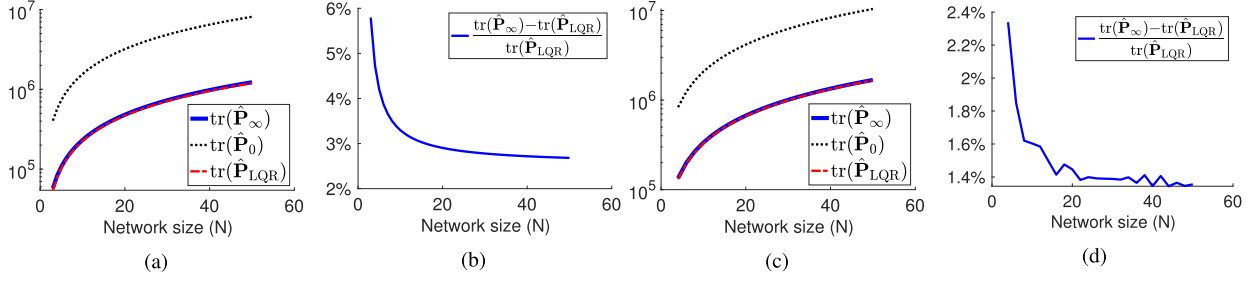


Fig. 3. Suboptimality of the distributed controller learned by Algorithm 1 for a network of N homogeneous plants with path graph structures in (a) and (b), and with random 3-regular graph structures in (c) and (d).

respect to the number of nodes in a path graph with different number of nodes. Fig. 3(a) compares the cost associated with our design $\hat{\mathbf{P}}_\infty$ against the cost of the initial controller $\hat{\mathbf{P}}_1$ and the (infeasible) LQR controller $\hat{\mathbf{P}}_{\text{LQR}}$. Fig. 3(b) illustrates the evolution of the normalized suboptimality of our proposed algorithm (with respect to the infeasible LQR controller) as a function of number of nodes in the corresponding graphs. Fig. 3(c) and (d) shows similar results for the random 3-regular graph topologies with even number of nodes.

As can be validated from Fig. 3, the cost associated with our final proposed distributed controller has significantly improved the optimality of the initial controller. In particular, the normalized suboptimality errors of our final design are less than 6% and 2.4% for path graph and random 3-regular graph topologies, respectively. Furthermore, this normalized error generally decreases as the number of nodes in the corresponding graphs increases.

VI. CONCLUDING REMARKS

In this article, we have proposed the D2SPI algorithm as an efficient model-free distributed control synthesis process for potentially high-dimensional network of homogeneous linear systems. D2SPI is built upon a construction referred to as Patterned monoid that facilitates exploiting network symmetries for policy synthesis consistent with the underlying network. Such symmetries allow a data collection procedure during the learning phase (with temporary additional links) for a smaller portion of the network. Using data collection on this smaller subnetwork, we are then able to synthesize a distributed feedback mechanism for the entire system—even during the learning phase. Moreover, D2SPI builds upon parameter estimation techniques that represent an end-to-end policy prediction directly from the observed data. Extension of the setup proposed for D2SPI involves heterogeneous networked systems, which is currently being examined as part of our future work.

APPENDIX A ANALYSIS AND PROOFS

In this section, we provide the building blocks needed for the proofs and analysis of our algorithm. We first provide some insights on how the setup is connected to the classic model-based LQR machinery and some previously established results that we leverage from the literature. The main proofs then follows.

A. Underlying Model of the Subsystem $\mathcal{G}_d$

The configuration of the synthesis problem in D2SPI intertwines an online recursion on the subsystem corresponding to $\mathcal{G}_d$ and the original system $\mathcal{G}$. In particular, during the learning phase, considering the same cost structure and problem parameters as in (3)—but for the completed subgraph $\mathcal{G}_{d,\text{learn}}$ —results in the $(\tilde{\mathbf{A}}, \tilde{\mathbf{B}}, \tilde{\mathbf{Q}}, \tilde{\mathbf{R}})$ parameters, as defined in Theorem 2. Then, similarly from (1), the dynamics of the subgraph $\mathcal{G}_{d,\text{learn}}$ assumes the form

$$\tilde{\mathbf{x}}_{t+1} = \tilde{\mathbf{A}}\tilde{\mathbf{x}}_t + \tilde{\mathbf{B}}\tilde{\mathbf{u}}_t \quad (4)$$

where $\tilde{\mathbf{x}}_t$ and $\tilde{\mathbf{u}}_t$ are formed from the concatenation of state and control signals in $\mathcal{G}_{d,\text{learn}}$ —recall that $\tilde{\mathbf{u}}_t$ is also denoted by $\text{Policy}(\mathcal{V}_{\mathcal{G}_{d,\text{learn}}})|_t$ in the algorithm to emphasize the temporal implementation of a specific policy in Algorithm 2. From the Bellman equation [61] for the LQR problem with these parameters, the cost matrix $\tilde{\mathbf{P}}_k$ of $\mathcal{G}_{d,\text{learn}}$ is correlated with the one-step LQR cost as

$$\tilde{\mathbf{x}}_t^\top \tilde{\mathbf{P}}_k \tilde{\mathbf{x}}_t = \mathcal{R}(\tilde{\mathbf{x}}_t, \tilde{\mathbf{u}}_t) + \tilde{\mathbf{x}}_{t+1}^\top \tilde{\mathbf{P}}_k \tilde{\mathbf{x}}_{t+1} \quad (5)$$

where $\mathcal{R}(\tilde{\mathbf{x}}_t, \tilde{\mathbf{u}}_t) = \tilde{\mathbf{x}}_t^\top \tilde{\mathbf{Q}} \tilde{\mathbf{x}}_t + \tilde{\mathbf{u}}_t^\top \tilde{\mathbf{R}} \tilde{\mathbf{u}}_t$ and $\tilde{\mathbf{P}}_k$ satisfies the Lyapunov equation

$$\tilde{\mathbf{P}}_k = (\tilde{\mathbf{A}} + \tilde{\mathbf{B}}\tilde{\mathbf{K}}_k)^\top \tilde{\mathbf{P}}_k (\tilde{\mathbf{A}} + \tilde{\mathbf{B}}\tilde{\mathbf{K}}_k) + \tilde{\mathbf{Q}} + \tilde{\mathbf{K}}_k^\top \tilde{\mathbf{R}} \tilde{\mathbf{K}}_k \quad (6)$$

and $\tilde{\mathbf{K}}_k$ is the controller policy at iteration k . The dynamic programming solution to the LQR problem suggests a linear feedback form $\tilde{\mathbf{u}}_t = \tilde{\mathbf{K}}_k \tilde{\mathbf{x}}_t$ for the subsystem $\mathcal{G}_{d,\text{learn}}$ at each iteration. Combining (4) and (5) with some rearrangements results in

$$\begin{aligned} \tilde{\mathbf{x}}_t^\top \tilde{\mathbf{P}}_k \tilde{\mathbf{x}}_t &= \tilde{\mathbf{z}}_t^\top \begin{bmatrix} \tilde{\mathbf{Q}} + \tilde{\mathbf{A}}^\top \tilde{\mathbf{P}}_k \tilde{\mathbf{A}} & \tilde{\mathbf{A}}^\top \tilde{\mathbf{P}}_k \tilde{\mathbf{B}} \\ \tilde{\mathbf{B}}^\top \tilde{\mathbf{P}}_k \tilde{\mathbf{A}} & \tilde{\mathbf{R}} + \tilde{\mathbf{B}}^\top \tilde{\mathbf{P}}_k \tilde{\mathbf{B}} \end{bmatrix} \tilde{\mathbf{z}}_t \\ &=: \tilde{\mathbf{z}}_t^\top \begin{bmatrix} [\tilde{\mathbf{H}}_k]_{11} & [\tilde{\mathbf{H}}_k]_{12} \\ [\tilde{\mathbf{H}}_k]_{21} & [\tilde{\mathbf{H}}_k]_{22} \end{bmatrix} \tilde{\mathbf{z}}_t = \tilde{\mathbf{z}}_t^\top \tilde{\mathbf{H}}_k \tilde{\mathbf{z}}_t \end{aligned} \quad (7)$$

where $\tilde{\mathbf{z}}_t = [\tilde{\mathbf{x}}_t^\top \tilde{\mathbf{u}}_t^\top]^\top$. Then, the following policy update (due to Hwer) is guaranteed to converge to the optimal LQR policy under the controllability assumption [21]

$$\tilde{\mathbf{K}}_{k+1} = -(\tilde{\mathbf{R}} + \tilde{\mathbf{B}}^\top \tilde{\mathbf{P}}_k \tilde{\mathbf{B}})^{-1} \tilde{\mathbf{B}}^\top \tilde{\mathbf{P}}_k \tilde{\mathbf{A}} = -[\tilde{\mathbf{H}}_k]_{22}^{-1} [\tilde{\mathbf{H}}_k]_{21} \quad (8)$$

which is also reconstructed by information in $\tilde{\mathbf{H}}_k$. Furthermore, the cost matrix in (6) can also be reconstructed by the same information

$$\tilde{\mathbf{P}}_k = [\tilde{\mathbf{H}}_k]_{11} + [\tilde{\mathbf{H}}_k]_{12}\tilde{\mathbf{K}}_k + \tilde{\mathbf{K}}_k^\top[\tilde{\mathbf{H}}_k]_{21} + \tilde{\mathbf{K}}_k^\top[\tilde{\mathbf{H}}_k]_{22}\tilde{\mathbf{K}}_k. \quad (9)$$

Hence, $\tilde{\mathbf{H}}_k$ provides the required information to perform both policy update (8) and policy evaluation steps (9) in a policy iteration algorithm. We will see that because of the particular structure of our setup, $\tilde{\mathbf{H}}_k$ enjoys a special block pattern captured by the proposed Patterned monoid, justifying the recovery of the block matrices X_1, X_2, Y_1, Y_2, Z_1 , and Z_2 from $\tilde{\mathbf{H}}_k$ in line 11 of D2SPI. D2SPI leverages this idea to implicitly learn $\tilde{\mathbf{H}}_k$ from data (by adapting the idea of [19]) and exploit these matrix blocks in order to find an efficient suboptimal solution to the main distributed problem in (3). Here, in addition to the policy update from data as in (8), we show that the same information can be used to also learn a gain margin directly from data (see Proposition 3). This gain margin is then used to guarantee the stability of the entire network (see Theorem 1).

Finally, for technical reasons, recall that the infinite-horizon state-feedback LQR problem with parameters (A, B, Q, R) can be cast as the minimization of

$$f_\Sigma(K) := \text{tr}[P_K \Sigma] \quad (10)$$

over the static stabilizing policy K , for some initial state distribution with covariance $\Sigma \succ 0$, where P_K is cost matrix associated with K satisfying the following Lyapunov equation [16], [17], [62]:

$$P_K = (A + BK)^\top P_K (A + BK) + Q + K^\top R K.$$

Herein, we set $\Sigma = \mathbf{I}$ and consider $f_1(K)$.

B. Technical Observations and Main Proofs

In the remainder of this section, we first restate some well-known technical facts to make this article self-contained and then propose a few additional algebraic facts for our analysis whose proofs are deferred to Appendix B. We then continue with the proof of the main results.

Lemma 1: The following relations hold:

- 1) (see [63]) When $X \succ 0$

$$M^\top X N + N^\top X M \succeq - \left(a M^\top X M + \frac{1}{a} N^\top X N \right)$$

$$M^\top X N + N^\top X M \preceq a M^\top X M + \frac{1}{a} N^\top X N$$

where $M, N \in \mathbb{R}^{n \times m}$ with $n \geq m$ and $a > 0$.

- 2) (see [64, Lyapunov equation]) Suppose that $A \in \mathbb{R}^{n \times n}$ has spectral radius less than 1, i.e., $\rho(A) < 1$. Then, $A^\top X A + Q - X = 0$ has a unique solution, and $X = \sum_{j=0}^{\infty} (A^\top)^j Q A^j$. In this case, if $Q \succ 0$, then $X \succ 0$.
- 3) (see [63, Block matrix inverse formula (0.8.5.6)]) The following identity holds for matrices A, B, C , and D

with compatible dimensions:

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix}^{-1} = \begin{bmatrix} H^{-1} & -H^{-1} B D^{-1} \\ -D^{-1} C H^{-1} & D^{-1} + D^{-1} C H^{-1} B D^{-1} \end{bmatrix}$$

where D and $H = A - B D^{-1} C$ are invertible.

- 4) (Matrix inversion lemma [65]) The following identity holds:

$$(A + U C V)^{-1} = A^{-1} - A^{-1} U (C^{-1} + V A^{-1} U)^{-1} V A^{-1}$$

for matrices A, U, C , and V with compatible dimensions where A, C , and $A + U C V$ are invertible.

Finally, we provide the main technical lemma that streamlines the properties of the Patterned monoid under algebraic manipulation, which will be frequently used in the proofs of Proposition 2 and Theorem 2.

Lemma 2: Suppose $\mathbf{N}_r \in \text{PM}(r \times n, \mathbb{R})$ for some n and $r \geq 2$ such that $\mathbf{N}_r = \mathbf{I}_r \otimes (A - B) + \mathbf{1}_r \mathbf{1}_r^\top \otimes B$, for some $A \in \text{GL}(n, \mathbb{R}) \cap \mathbb{S}^{n \times n}$ and $B \in \mathbb{S}^{n \times n}$. Then, the following hold.

- 1) $\det(\mathbf{N}_r) = \det(A - B)^{r-1} \det(A + (r-1)B)$.
- 2) If $\mathbf{N}_r \succ 0$, then we have $A + (\ell-1)B \succ 0$ for all $\ell = 0, 1, \dots, r$. Furthermore, $A - \ell B (A + (\ell-1)B)^{-1} B$ is invertible for $\ell = 1, 2, \dots, r-1$.
- 3) If $\mathbf{N}_r \succ 0$, then $\mathbf{N}_r^{-1} \in \text{PM}(r \times n, \mathbb{R})$; in fact

$$\mathbf{N}_r^{-1} = \mathbf{I}_r \otimes (F_r + G_r) - \mathbf{1}_r \mathbf{1}_r^\top \otimes G_r$$

with F_r and G_r defined as

$$F_r = \left(A - (r-1)B (A + (r-2)B)^{-1} B \right)^{-1}$$

$$G_r = (A + (r-1)B)^{-1} B (A - B)^{-1}.$$

- 4) If also $\mathbf{M}_r \in \text{L}(r \times n, \mathbb{R})$, i.e., $\mathbf{M}_r = \mathbf{I}_r \otimes (C - D) + \mathbf{1}_r \mathbf{1}_r^\top \otimes D$ for some $C, D \in \mathbb{R}^{n \times n}$, then

$$\mathbf{N}_r \mathbf{M}_r = \mathbf{I}_r \otimes (A - B)(C - D)$$

$$+ \mathbf{1}_r \mathbf{1}_r^\top \otimes (B(C - D) + (A - B)D + rBD).$$

1) Proof of Proposition 1: For any Schur stable matrix A , and any symmetric positive-definite matrix Q , there exists a unique positive-definite solution P to the discrete Lyapunov equation described by $P = \sum_{j=0}^{\infty} (A^\top)^j Q A^j$ (see item 2 in Lemma 1). Note that $\text{PM}(r \times n, \mathbb{R}) \subset \mathbb{S}^{rn \times rn} \cap \text{L}(r \times n, \mathbb{R})$ by construction. Therefore, since $\text{PM}(r \times n, \mathbb{R})$ is a monoid and $\text{L}(r \times n, \mathbb{R})$ closed under matrix multiplication by Lemma 2, each summand falls in $\text{L}(r \times n, \mathbb{R})$ whenever $Q \in \text{PM}(r \times n, \mathbb{R})$. Also, as the infinite sum preserves the structure, $P \in \text{L}(r \times n, \mathbb{R})$. To show that $P \in \text{PM}(r \times n, \mathbb{R})$, it now suffices to note that $P \succ 0$ whenever $Q \succ 0$ because then every principle submatrix of P has to be positive definite and thus invertible. Conversely, if $P \in \text{PM}(r \times n, \mathbb{R})$, then $Q = P - A^\top P A$ also must lie in $\text{PM}(r \times n, \mathbb{R})$ as $Q \succ 0$. This completes the proof. $\square$

2) Proof of Proposition 2: At iteration k of the learning phase in Algorithm 1, the first claim is a direct consequence of the structure of $\mathcal{G}_{d,\text{learn}}$ during the learning phase, where $\mathcal{G}_{d,\text{learn}} = \mathcal{K}(\mathcal{G}_d)$ and hence $\mathbf{u}_i = (K_k - L_k)\mathbf{x}_i + L_k \sum_{j \in \mathcal{V}_{\mathcal{G}_{d,\text{learn}}}} \mathbf{x}_j$ for all $i \in \mathcal{V}_{\mathcal{G}_{d,\text{learn}}}$, which, in turn, results in

$\tilde{\mathbf{u}}_t = \tilde{\mathbf{K}}_k \tilde{\mathbf{x}}_t$ with $\tilde{\mathbf{K}}_k$ as claimed. The stability of the policy $\tilde{\mathbf{K}}_k$ for the pair $(\tilde{\mathbf{A}}, \tilde{\mathbf{B}})$ throughout the learning phase is then argued in Appendix A-B5 under Assumption 1.

Next, the cost matrix $\tilde{\mathbf{P}}_k$ associated with $\tilde{\mathbf{K}}_k$ satisfies the Lyapunov equation (6). We can verify that $\tilde{\mathbf{A}} + \tilde{\mathbf{B}}\tilde{\mathbf{K}}_k \in \mathcal{L}(d \times n, \mathbb{R})$, which is also Schur stable, and $\tilde{\mathbf{Q}} + \tilde{\mathbf{K}}_k^\top \tilde{\mathbf{R}}\tilde{\mathbf{K}}_k \in \mathcal{PM}(d \times n, \mathbb{R})$. Thus, by Proposition 1, we conclude that $\tilde{\mathbf{P}}_k \in \mathcal{PM}(d \times n, \mathbb{R})$. Therefore, let

$$\tilde{\mathbf{P}}_k = \mathbf{I}_d \otimes (P_1 - P_2) + \mathbf{1}\mathbf{1}^\top \otimes P_2 \quad (11)$$

for some P_1 and P_2 . Note that $\tilde{\mathbf{K}}_k$ is stabilizing and $\tilde{\mathbf{Q}} + \tilde{\mathbf{K}}_k^\top \tilde{\mathbf{R}}\tilde{\mathbf{K}}_k \succ 0$; therefore, $\tilde{\mathbf{P}}_k \succ 0$ from (6). But then, by item 2 in Lemma 2 and the structure of $\tilde{\mathbf{P}}_k$ from (11), we claim that $\Delta P_k := P_1 - P_2 \succ 0$. Next, one can also verify that

$$\begin{aligned} & (\tilde{\mathbf{A}} + \tilde{\mathbf{B}}\tilde{\mathbf{K}}_k)^\top \tilde{\mathbf{P}}_k (\tilde{\mathbf{A}} + \tilde{\mathbf{B}}\tilde{\mathbf{K}}_k) \\ &= \mathbf{I}_d \otimes (A_{\Delta K_k}^\top (\Delta P_k) A_{\Delta K_k}) + \mathbf{1}\mathbf{1}^\top \otimes (\star) \\ & \quad \tilde{\mathbf{Q}} + \tilde{\mathbf{K}}_k^\top \tilde{\mathbf{R}}\tilde{\mathbf{K}}_k \\ &= \mathbf{I}_d \otimes (Q_d + (\Delta K_k)^\top R(\Delta K_k)) + \mathbf{1}\mathbf{1}^\top \otimes (\star) \end{aligned}$$

where $\Delta K_k = K_k - L_k$, $A_{\Delta K_k} = A + B(\Delta K_k)$, $Q_d = Q_1 + dQ_2$, and $(\star)$ is hiding extra terms. But then, by (6) and (11), we obtain that ΔP_k must satisfy

$$\Delta P_k = A_{\Delta K_k}^\top (\Delta P_k) A_{\Delta K_k} + Q_d + \Delta K_k^\top R \Delta K_k \quad (12)$$

which itself is a Lyapunov equation. Finally, since $Q_d + \Delta K_k^\top R \Delta K_k \succ 0$ and $\Delta P_k \succ 0$, by the Lyapunov stability criterion, we conclude that $A_{\Delta K_k}$ is Schur stable. This completes the proof. $\square$

3) Proof of Proposition 3: Define the Lyapunov candidate function $V_k(\mathbf{x}_t) = \mathbf{x}_t^\top \Delta P_k \mathbf{x}_t$ with $\Delta P_k \succ 0$ as in (12) and where $\mathbf{x}_t$ contains the states of the closed-loop system $\mathbf{x}_{t+1} = (A + B(K_k - \alpha L_k))\mathbf{x}_t$ for some scalar α . We show that for the given choice of α , V_k is decreasing. We define

$$\Delta V_k(\mathbf{x}_t) := V_k(\mathbf{x}_{t+1}) - V_k(\mathbf{x}_t) = \mathbf{x}_t^\top \Gamma_k \mathbf{x}_t$$

where

$$\Gamma_k = (A_{\Delta K_k} + (1 - \alpha)L_k)^\top \Delta P_k (A_{\Delta K_k} + (1 - \alpha)L_k) - \Delta P_k.$$

Suppose $\alpha < 1$; then, from (12), we have

$$\begin{aligned} \Gamma_k &= -(Q_d + \Delta K_k^\top R \Delta K_k) + (1 - \alpha)^2 L_k^\top B^\top \Delta P_k B L_k \\ & \quad + (1 - \alpha) (A_{\Delta K_k}^\top \Delta P_k B L_k + L_k^\top B^\top \Delta P_k A_{\Delta K_k}) \\ &\leq -(Q_d + \Delta K_k^\top R \Delta K_k) + (1 - \alpha)^2 L_k^\top B^\top \Delta P_k B L_k \\ & \quad + (1 - \alpha) (a A_{\Delta K_k}^\top \Delta P_k A_{\Delta K_k} + (1/a) L_k^\top B^\top \Delta P_k B L_k) \\ &= -(Q_d + \Delta K_k^\top R \Delta K_k) + (1 - \alpha) a A_{\Delta K_k}^\top \Delta P_k A_{\Delta K_k} \\ & \quad + (1 - \alpha) (1/a + 1 - \alpha) L_k^\top B^\top \Delta P_k B L_k \end{aligned}$$

where the inequality holds for any $a > 0$ due to Lemma 1. Let $\beta = (1 - \alpha)/2$ and choose $a = \beta + \sqrt{\beta^2 + 1}$. Then, $(1/a + 1 - \alpha) = a$ and thus

$$\lambda_{\max}(\Gamma_k) \leq -\lambda_{\min} [Q_d + \Delta K_k^\top R \Delta K_k]$$

$$+ (1 - \alpha) a \lambda_{\max} [A_{\Delta K_k}^\top \Delta P_k A_{\Delta K_k} + L_k^\top B^\top \Delta P_k B L_k].$$

Now, using the parameters (13) constructing the blocks of (7), we obtain

$$\begin{aligned} L_k^\top B^\top \Delta P_k B L_k &= L_k^\top (\Delta Y - R) L_k, \quad \text{and} \\ A_{\Delta K_k}^\top \Delta P_k A_{\Delta K_k} &= \Delta X - Q_d + \Delta K_k^\top \Delta Z + \Delta Z^\top \Delta K_k \\ & \quad + \Delta K_k^\top (\Delta Y - R) \Delta K_k =: \Xi_k. \end{aligned}$$

Thus, the latter bound can be obtained completely from data as

$$\lambda_{\max}(\Gamma_k) \leq ((1 - \alpha)a - \gamma_k) \lambda_{\max} [\Xi_k + L_k^\top (\Delta Y - R) L_k]$$

with

$$\gamma_k = \lambda_{\min} [Q_d + \Delta K_k^\top R \Delta K_k] / \lambda_{\max} [\Xi_k + L_k^\top (\Delta Y - R) L_k]$$

which coincides with updates in line 13 of Algorithm 1.

Finally, from the hypothesis $1 - \tau_k < \alpha < 1$ with $\tau_k = \sqrt{\gamma_k^2 / (1 + \gamma_k)}$, we obtain that $\gamma_k^2 - 4\beta^2 \gamma_k - 4\beta^2 > 0$. However, since $\gamma_k > 0$, this second-order term in γ_k is positive only if

$$\gamma_k > 2\beta^2 + 2\beta\sqrt{\beta^2 + 1} = 2\beta(\beta + \sqrt{\beta^2 + 1}) = (1 - \alpha)a.$$

Therefore, $\Delta V_k(\mathbf{x}_t) < 0$ for $1 - \tau_k < \alpha < 1$. Similar reasoning for $1 < \alpha < 1 + \tau_k$ also shows that $\Delta V_k(\mathbf{x}_t) < 0$, which completes the proof. $\square$

4) Proof of Theorem 1: From Definition 1 (according to a consistent choice of labeling of the nodes so that the last d nodes are chosen as $\mathcal{G}_d$) and Proposition 2, the feedback policy in (line 9) of Algorithm 1 can be cast in the compact form

$$\begin{aligned} \text{Policy}_k(\mathcal{V}_\mathcal{G}) &= \begin{bmatrix} \text{Policy}_k(\mathcal{V}_{\mathcal{G} \setminus \mathcal{G}_d, \text{learn}}) \\ \text{Policy}_k(\mathcal{V}_{\mathcal{G}_d, \text{learn}}) \end{bmatrix} \\ &= \begin{bmatrix} \hat{\mathbf{K}}_{\mathcal{G} \setminus \mathcal{G}_d} & \hat{\mathbf{L}}_{\mathcal{G} \setminus \mathcal{G}_d} \\ \mathbf{0} & \tilde{\mathbf{K}}_k \end{bmatrix} \begin{bmatrix} \text{State}_k(\mathcal{V}_{\mathcal{G} \setminus \mathcal{G}_d, \text{learn}}) \\ \text{State}_k(\mathcal{V}_{\mathcal{G}_d, \text{learn}}) \end{bmatrix} \\ &=: \hat{\mathbf{K}}_k \text{State}_k(\mathcal{G}) \end{aligned}$$

where $\hat{\mathbf{L}}_{\mathcal{G} \setminus \mathcal{G}_d} := \frac{\tau_k}{d-1} [\mathcal{A}_\mathcal{G}]_{12} \otimes L_k$, $\tilde{\mathbf{K}}_k := \mathbf{I}_d \otimes (K_k - L_k) + \mathbf{1}_d \mathbf{1}_d^\top \otimes L_k$, $\hat{\mathbf{K}}_{\mathcal{G} \setminus \mathcal{G}_d} := \mathbf{I}_{N-d} \otimes K_k - (\mathbf{I}_{N-d} - \frac{\tau_k}{d-1} \mathcal{A}_{\mathcal{G} \setminus \mathcal{G}_d}) \otimes L_k$, $\mathcal{A}_\mathcal{G}$ denotes the adjacency matrix of $\mathcal{G}$, and $[\mathcal{A}_\mathcal{G}]_{12}$ is its submatrix capturing the interconnection of $\mathcal{G} \setminus \mathcal{G}_d$ and $\mathcal{G}_d$:

$$\mathcal{A}_\mathcal{G} = \begin{bmatrix} \mathcal{A}_{\mathcal{G} \setminus \mathcal{G}_d} & [\mathcal{A}_\mathcal{G}]_{12} \\ * & \mathcal{A}_{\mathcal{G}_d} \end{bmatrix}.$$

Note that the structure of $\hat{\mathbf{K}}_k$ emanates from the fact that the information exchange is unidirectional during the learning phase. Now, consider the closed-loop system of $\mathcal{G}$ as

$$\begin{aligned} \hat{\mathbf{A}}_\mathcal{G}|_{cl} &:= \hat{\mathbf{A}} + \hat{\mathbf{B}}\hat{\mathbf{K}}_k \\ &= \begin{bmatrix} \mathbf{I}_{N-d} \otimes A + (\mathbf{I}_{N-d} \otimes B) \hat{\mathbf{K}}_{\mathcal{G} \setminus \mathcal{G}_d} & (\mathbf{I}_{N-d} \otimes B) \hat{\mathbf{L}}_{\mathcal{G} \setminus \mathcal{G}_d} \\ \mathbf{0} & \tilde{\mathbf{A}}_{\tilde{\mathbf{K}}_k} \end{bmatrix} \end{aligned}$$

where $\tilde{\mathbf{A}}_{\tilde{\mathbf{K}}_k} := \tilde{\mathbf{A}} + \tilde{\mathbf{B}}\tilde{\mathbf{K}}_k$ is the closed-loop system of $\mathcal{G}_d$. Define $S = \mathbf{I}_{N-d} - \frac{\tau_k}{d-1} \mathcal{A}_{\mathcal{G} \setminus \mathcal{G}_d}$, and let J be the Jordan form of S according to the similarity transformation $\mathcal{T} \in \mathbb{R}^{(N-d) \times (N-d)}$

such that $\mathcal{T}S\mathcal{T}^{-1} = J$. Now, consider the following similarity transformation of $\hat{\mathbf{A}}_{\mathcal{G}|cl}$:

$$\begin{aligned} & \begin{bmatrix} \mathcal{T} \otimes \mathbf{I}_n & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_d \otimes \mathbf{I}_n \end{bmatrix} \left(\hat{\mathbf{A}}_{\mathcal{G}|cl} \right) \begin{bmatrix} \mathcal{T} \otimes \mathbf{I}_n & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_d \otimes \mathbf{I}_n \end{bmatrix}^{-1} \\ &= \begin{bmatrix} \mathbf{I}_{N-d} \otimes (A + BK_k) - J \otimes BL_k & * \\ \mathbf{0} & \tilde{\mathbf{A}}_{\tilde{\mathbf{K}}_k} \end{bmatrix}. \end{aligned}$$

Note that $\mathbf{I}_{N-d} \otimes (A + BK_k) - J \otimes BL_k$ is a block upper triangular matrix whose diagonal blocks are equal to $A + B(K_k - \lambda_i(S)L_k)$ for $i = 1, \dots, N-d$. We already know from Proposition 2 that $\tilde{\mathbf{A}}_{\tilde{\mathbf{K}}_k}$ is Schur stable. Hence, in order to show that $\hat{\mathbf{A}}_{\mathcal{G}|cl}$ is Schur stable, it suffices to show that $\rho(A + B(K - \lambda_i(S)L)) < 1$ for $i = 1, \dots, N-d$. Recall that $|\lambda_i(\mathcal{A}_{\mathcal{G}})| \leq d_{\max}$ [66]; thus, by definition of S and the fact that $d_{\max} = d-1$, we conclude that $|\lambda_i(S) - 1| \leq \tau_k$. The claim now follows directly from Proposition 3. $\square$

5) Proof of Theorem 2: At iteration k of the learning phase in Algorithm 1, consider $\tilde{\mathbf{H}}_k$ and its corresponding blocks, as defined in (7). First, we consider the structure of the stabilizing feedback policy $\tilde{\mathbf{K}}_k$, as shown in Proposition 2, together with that of system parameters $(\tilde{\mathbf{A}}, \tilde{\mathbf{B}}, \tilde{\mathbf{Q}}, \tilde{\mathbf{R}})$, and apply Lemma 2 and Proposition 1 to conclude that $[\tilde{\mathbf{H}}_k]_{11}, [\tilde{\mathbf{H}}_k]_{22} \in \text{PM}(d \times n, \mathbb{R})$ and $[\tilde{\mathbf{H}}_k]_{21} \in \text{L}(d \times n, \mathbb{R})$. Thus, we get

$$\begin{aligned} [\tilde{\mathbf{H}}_k]_{11} &= \mathbf{I}_d \otimes (X_1 - X_2) + \mathbf{1}\mathbf{1}^\top \otimes X_2 \\ [\tilde{\mathbf{H}}_k]_{22} &= \mathbf{I}_d \otimes (Y_1 - Y_2) + \mathbf{1}\mathbf{1}^\top \otimes Y_2 \\ [\tilde{\mathbf{H}}_k]_{21} &= \mathbf{I}_d \otimes (Z_1 - Z_2) + \mathbf{1}\mathbf{1}^\top \otimes Z_2 \end{aligned}$$

which coincides with the recovery of X_i, Y_i , and Z_i for $i = 1, 2$ in line 11 of Algorithm 1. We can also unravel the structure of block matrices constructing $\tilde{\mathbf{H}}_k$ and obtain that

$$\begin{aligned} X_1 &= Q_1 + (d-1)Q_2 + A^\top P_1 A, & Y_2 &= B^\top P_2 B \\ X_2 &= -Q_2 + A^\top P_2 A, & Z_1 &= B^\top P_1 A \\ Y_1 &= R + B^\top P_1 B, & Z_2 &= B^\top P_2 A \end{aligned} \quad (13)$$

with P_1 and P_2 as in (11). Similarly, by Lemma 2, we obtain

$$[\tilde{\mathbf{H}}_k]_{22}^{-1} = \mathbf{I}_d \otimes (F - G) + \mathbf{1}\mathbf{1}^\top \otimes G$$

where (by item 3 in Lemma 2) F and G must satisfy

$$\begin{aligned} F^{-1} &= Y_1 - (d-1)Y_2(Y_1 + (d-2)Y_2)^{-1}Y_2 \\ G &= (Y_1 + (d-1)Y_2)^{-1}Y_2(Y_1 - Y_2)^{-1} \end{aligned}$$

which coincides with the definitions in line 12 of Algorithm 1. Finally, by definition of K_{k+1} and L_{k+1} in line 12, and item 4 in Lemma 2, one can verify that $\tilde{\mathbf{K}}_{k+1} = -[\tilde{\mathbf{H}}_k]_{22}^{-1}[\tilde{\mathbf{H}}_k]_{21}$, which coincides with the policy iteration in Hewer's algorithm [21] for the system in $\mathcal{G}_{d,\text{learn}}$ (see also Appendix A-A). Note that by assumption, the pair (A, B) is controllable, so is the system $(\tilde{\mathbf{A}}, \tilde{\mathbf{B}})$ in $\mathcal{G}_{d,\text{learn}}$. Therefore, these updates are guaranteed to remain stabilizing and converge to the claimed optimal LQR policy $\tilde{\mathbf{K}}^*$, provided that we have access to the true parameters $\mathbf{H}_k$.

Next, consider the LQR cost $f_1(\tilde{\mathbf{K}})$ as in (10) but for the problem parameters $(\tilde{\mathbf{A}}, \tilde{\mathbf{B}}, \tilde{\mathbf{Q}}, \tilde{\mathbf{R}})$. For completing the proof of convergence, it is left to argue that there exists a large enough integer C such that, at each iteration k of the learning phase, the RLS in Algorithm 2 provides a more accurate estimation, denoted by θ_k° , of the true parameters $\theta_k = \text{vech}(\tilde{\mathbf{H}}_k)$, and the LQR cost $f_1(\tilde{\mathbf{K}}) = \text{tr}(\tilde{\mathbf{P}}_{\tilde{\mathbf{K}}})$ decreases. This claim essentially follows by [19, Th. 1], which we try to summarize for completeness. For that, at iteration k , let us denote the policy obtained using the estimated parameters by $\tilde{\mathbf{K}}_k^\circ$, which, in turn, estimates the true policy $\tilde{\mathbf{K}}_k$. We then define a ‘‘Lyapunov’’ function candidate

$$s_k := f_1(\tilde{\mathbf{K}}_{k-1}^\circ) + \|\theta_{k-2} - \theta_{k-2}^\circ\|.$$

Following the same induction reasoning as that of [19, Th. 1] and under PE input, there exists an integer C such that

$$s_{k+1} = s_k - \varepsilon_1(C)\|\theta_{k-1} - \theta_{k-1}^\circ\| - \varepsilon_2(C)\|\tilde{\mathbf{K}}_k - \tilde{\mathbf{K}}_{k-1}^\circ\|^2$$

for some positive constants $\varepsilon_1(C)$ and $\varepsilon_2(C)$ that are independent of k . But then, $s_{k+1} \leq s_k$ and

$$\begin{aligned} \varepsilon_1(C) \sum_{k=2}^{\infty} \|\theta_{k-1} - \theta_{k-1}^\circ\| &\leq s_1; \\ \varepsilon_2(C) \sum_{k=2}^{\infty} \|\tilde{\mathbf{K}}_k - \tilde{\mathbf{K}}_{k-1}^\circ\|^2 &\leq s_1. \end{aligned}$$

Also, s_1 is bounded as $\tilde{\mathbf{K}}_1$ stabilizes $(\tilde{\mathbf{A}}, \tilde{\mathbf{B}})$. This guarantees that, first, $\tilde{\mathbf{K}}_k^\circ$ remains stabilizing as $f_1(\tilde{\mathbf{K}}_{k-1}^\circ) \leq s_0$ and $\tilde{\mathbf{Q}} \succ 0$; second, the estimates θ_{k-1}° become more accurate; and third, $\tilde{\mathbf{K}}_k^\circ \rightarrow \tilde{\mathbf{K}}^*$ as $\tilde{\mathbf{K}}_{k+1} \rightarrow \tilde{\mathbf{K}}^*$ at $k \rightarrow \infty$ [21]. This completes the proof. $\square$

6) Proof of Theorem 3: Consider the LQR cost $f_1(\hat{\mathbf{K}})$ as in (10) but for the problem parameters $(\hat{\mathbf{A}}, \hat{\mathbf{B}}, \hat{\mathbf{Q}}, \hat{\mathbf{R}})$. In the ‘‘unstructured’’ case (i.e., ignoring the constraint $K \in \mathcal{U}_{m,n}^N(\mathcal{G})$), we know that the optimal LQR cost matrix for the entire networked system satisfies [67]

$$\hat{\mathbf{P}}_{\text{lqr}}^* = \hat{\mathbf{A}}_{\hat{\mathbf{K}}_{\text{lqr}}}^\top \hat{\mathbf{P}}_{\text{lqr}} \hat{\mathbf{A}}_{\hat{\mathbf{K}}_{\text{lqr}}} + \hat{\mathbf{K}}_{\text{lqr}}^\top \hat{\mathbf{R}} \hat{\mathbf{K}}_{\text{lqr}} + \hat{\mathbf{Q}} \quad (14)$$

where $\hat{\mathbf{K}}_{\text{lqr}} = \arg \min_K f_1(K)$. Moreover, the cost matrix $\hat{\mathbf{P}}^*$ associated with the structured policy $\hat{\mathbf{K}}^*$ learned by Algorithm 1 satisfies

$$\hat{\mathbf{P}}^* = \mathbf{A}_{\hat{\mathbf{K}}^*}^\top \hat{\mathbf{P}}^* \mathbf{A}_{\hat{\mathbf{K}}^*} + \hat{\mathbf{K}}^{*\top} \hat{\mathbf{R}} \hat{\mathbf{K}}^* + \hat{\mathbf{Q}} \quad (15)$$

where $\mathbf{A}_{\hat{\mathbf{K}}^*} = \hat{\mathbf{A}} + \hat{\mathbf{B}}\hat{\mathbf{K}}^*$ and $\hat{\mathbf{K}}^* \in \mathcal{U}_{m,n}^N(\mathcal{G})$. Finally, let $\hat{\mathbf{K}}_{\text{struc}}^* \in \arg \min_{K \in \mathcal{U}_{m,n}^N(\mathcal{G})} f_1(K)$ denote a ‘‘structured’’ stabilizing optimal LQR policy, which is associated with the cost matrix $\hat{\mathbf{P}}_{\text{struc}}^*$. We know that such a policy exists since the smooth cost is lower-bounded and $\hat{\mathbf{K}}_1 = \mathbf{I}_N \otimes \mathbf{K}_1 \in \mathcal{U}_{m,n}^N(\mathcal{G})$ is a feasible point of this optimization—as $\mathbf{K}_1$ is assumed to be stabilizing for the single pair (A, B) . Therefore

$$\text{tr}[\hat{\mathbf{P}}_{\text{lqr}}^*] = f_1(\hat{\mathbf{K}}_{\text{lqr}}^*) \leq f_1(\hat{\mathbf{K}}_{\text{struc}}^*) \leq f_1(\hat{\mathbf{K}}^*) = \text{tr}[\hat{\mathbf{P}}^*]$$

where the last inequality above follows by the fact that $\hat{\mathbf{K}}^*$ is a feasible solution to the structured problem by construction,

i.e., $\hat{\mathbf{K}}^* \in \mathcal{U}_{m,n}^N(\mathcal{G})$. Therefore, $0 \leq \text{gap}(\hat{\mathbf{K}}^*) \leq \text{tr}[\hat{\mathbf{P}}^* - \hat{\mathbf{P}}_{\text{lqr}}^*]$. But then, one can obtain from (14), (15), and some algebraic manipulation that

$$\hat{\mathbf{P}}^* - \hat{\mathbf{P}}_{\text{lqr}}^* = \mathbf{A}_{\hat{\mathbf{K}}_{\text{lqr}}^*}^\top \left(\hat{\mathbf{P}}^* - \hat{\mathbf{P}}_{\text{lqr}} \right) \mathbf{A}_{\hat{\mathbf{K}}_{\text{lqr}}^*} + \mathbf{M}'$$

where

$$\mathbf{M}' = \mathbf{A}_{\hat{\mathbf{K}}^*}^\top \hat{\mathbf{P}}^* \mathbf{A}_{\hat{\mathbf{K}}^*} - \mathbf{A}_{\hat{\mathbf{K}}_{\text{lqr}}^*}^\top \hat{\mathbf{P}}^* \mathbf{A}_{\hat{\mathbf{K}}_{\text{lqr}}^*} + \hat{\mathbf{K}}^{*\top} \hat{\mathbf{R}} \hat{\mathbf{K}}^* - \hat{\mathbf{K}}_{\text{lqr}}^\top \hat{\mathbf{R}} \hat{\mathbf{K}}_{\text{lqr}}.$$

Since $\hat{\mathbf{P}}^* - \hat{\mathbf{P}}_{\text{lqr}}^* \succ 0$ and $\hat{\mathbf{A}}_{\hat{\mathbf{K}}_{\text{lqr}}^*}$ is contractible by the hypothesis, from the first part and [68, Th. 1], we obtain

$$\text{gap}(\hat{\mathbf{K}}^*) \leq \frac{\text{tr}(\mathbf{M}')}{1 - \sigma_{\max}^2(\hat{\mathbf{A}}_{\hat{\mathbf{K}}_{\text{lqr}}^*})} = \frac{\text{tr}(\mathbf{M})}{1 - \sigma_{\max}^2(\hat{\mathbf{A}}_{\hat{\mathbf{K}}_{\text{lqr}}^*})}$$

where the last equality follows by the cyclic permutation property of trace and definition of $\mathbf{A}_{\hat{\mathbf{K}}^*}$. $\square$

APPENDIX B PROOF OF LEMMA 2

First, we show that the following algebraic identities hold, which will be used in the proof of Lemma 2.

Lemma 3: Suppose that A and B are symmetric matrices such that A , $A - B$, and $A + (n - 1)B$ are all invertible for some integer n . Then, the following relations hold.

- 1) $(A + nB)(A + (n - 1)B)^{-1}(A - B) = A - nB(A + (n - 1)B)^{-1}B$.
- 2) $(A + nB)(A + (n - 1)B)^{-1}(A - B) = (A - B)(A + (n - 1)B)^{-1}(A + nB)$.
- 3) $(A + nB)(A - B)^{-1}B = B(A - B)^{-1}(A + nB)$.

Proof: These claims follow by the following algebraic manipulations: First

$$\begin{aligned} & (A + nB)(A + (n - 1)B)^{-1}(A - B) \\ &= ((A + (n - 1)B) + B)(A + (n - 1)B)^{-1}(A - B) \\ &= \left(I + B(A + (n - 1)B)^{-1} \right) (A - B) \\ &= A - B + B(A + (n - 1)B)^{-1}A - B(A + (n - 1)B)^{-1}B \\ &= A + B \left((A + (n - 1)B)^{-1}A - I \right) - B(A + (n - 1)B)^{-1}B \\ &= A + B(A + (n - 1)B)^{-1}(A - (A + (n - 1)B)) \\ &\quad - B(A + (n - 1)B)^{-1}B \\ &= A - (n - 1)B(A + (n - 1)B)^{-1}B - B(A + (n - 1)B)^{-1}B \\ &= A - nB(A + (n - 1)B)^{-1}B. \end{aligned}$$

Second

$$\begin{aligned} & (A + nB)(A + (n - 1)B)^{-1}(A - B) \\ &= -n(A - B)(A + (n - 1)B)^{-1}(A - B) \\ &\quad + (n + 1)A(A + (n - 1)B)^{-1}(A - B) \\ &= -n(A - B)(A + (n - 1)B)^{-1}(A - B) \end{aligned}$$

$$\begin{aligned} & + (n + 1)(I + (n - 1)BA^{-1})^{-1}(A - B) \\ &= -n(A - B)(A + (n - 1)B)^{-1}(A - B) \\ &\quad + (n + 1)(A - B)((I + (n - 1)BA^{-1})(A - B))^{-1}(A - B) \\ &= -n(A - B)(A + (n - 1)B)^{-1}(A - B) \\ &\quad + (n + 1)(A - B)((A - B)(I + (n - 1)A^{-1}B))^{-1}(A - B) \\ &= -n(A - B)(A + (n - 1)B)^{-1}(A - B) \\ &\quad + (n + 1)(A - B)(A + (n - 1)B)^{-1}A \\ &= (A - B)(A + (n - 1)B)^{-1}(A + nB). \end{aligned}$$

Finally

$$\begin{aligned} & (A + nB)(A - B)^{-1}B \\ &= (A - B + (n + 1)B)(A - B)^{-1}B \\ &= (I + (n + 1)B(A - B)^{-1})B \\ &= B(I + (n + 1)(A - B)^{-1}B) \\ &= B(A - B)^{-1}(A + nB). \end{aligned} \quad \square$$

Proof of Lemma 2

- 1) We prove the claim by induction. First, note that both $\mathbf{N}_r$ and its principle submatrix A are invertible. For $r = 2$, by the Schur complement of $\mathbf{N}_2$, we get

$$\begin{aligned} \det(\mathbf{N}_2) &= \det(A) \det(A - BA^{-1}B) \\ &= \det(A) \det(I - BA^{-1}) \det(I + BA^{-1}) \det(A) \\ &= \det(A - B) \det(A + B). \end{aligned}$$

Now, suppose that the claim holds for $r = p$. Then, for $r = p + 1$, similarly by the Schur complement, we get

$$\begin{aligned} \det(\mathbf{N}_{p+1}) &= \det(A) \det(\mathbf{N}_p - \mathbb{1}\mathbb{1}^\top \otimes BA^{-1}B) \\ &= \det(A) \det(A - B)^{p-1} \\ &\quad \cdot \det(A - BA^{-1}B + (p - 1)(B - BA^{-1}B)) \\ &= \det(A) \det(A - B)^{p-1} \\ &\quad \cdot \det(A - B + pBA^{-1}(A - B)) \\ &= \det(A) \det(A - B)^p \det(I + pBA^{-1}) \\ &= \det(A - B)^p \det(A + pB) \end{aligned}$$

where the second equality follows by applying the induction hypothesis to $\mathbf{N}_p - \mathbb{1}\mathbb{1}^\top \otimes BA^{-1}B$ and some algebraic manipulation. This completes the proof.

- 2) From item 1 of Lemma 2, we have

$$\begin{aligned} & \det(\mathbf{N}_r - \lambda I_r \otimes I) \\ &= \det(A - \lambda I - B)^{r-1} \det(A - \lambda I + (r - 1)B) \end{aligned}$$

implying that the spectrum of $\mathbf{N}_r$ coincides with that of $A - B$ and $A + (r - 1)B$ —modulo algebraic multiplicities. Hence, $\mathbf{N}_r \succ 0$ results in $A - B \succ 0$ and

$A + (r - 1)B \succ 0$. Furthermore, $\mathbf{N}_r \succ 0$ if and only if its principal submatrices are positive definite. Therefore, by applying the latter result to principal submatrices, we claim that $A + \ell B \succ 0$ for $\ell = 0, \dots, r - 2$. Finally, from item 1 of Lemma 3, for $\ell = 1, \dots, r - 1$, we have

$$\begin{aligned} A - \ell B (A + (\ell - 1)B)^{-1} B \\ = (A + \ell B) (A + (\ell - 1)B)^{-1} (A - B) \end{aligned}$$

which, by the first part of this claim, is invertible as a multiplication of invertible matrices.

- 3) Since $\mathbf{N}_r$ and A are invertible, the Schur complement $\mathbf{N}_{r-1} - L_{r-1} A^{-1} L_{r-1}^\top$ is also invertible where $L_{r-1} = \mathbb{1}_{r-1} \otimes B$. We prove the claim by induction on r . For $r = 2$, by [63, Block matrix inverse formula (0.8.5.6)], we have

$$\mathbf{N}_2^{-1} = \begin{bmatrix} H^{-1} & -H^{-1} B A^{-1} \\ -A^{-1} B H^{-1} & A^{-1} + A^{-1} B H^{-1} B A^{-1} \end{bmatrix}$$

where $H = A - B A^{-1} B$ is the Schur complement of A . By [63, Woodbury inversion formula (0.7.4.1)], $H^{-1} = A^{-1} + A^{-1} B H^{-1} B A^{-1}$, establishing the recurrence of diagonal blocks. Also, $\mathbf{N}_2$ is symmetric, so is $\mathbf{N}_2^{-1}$ and thus establishing that $\mathbf{N}_2 \in \text{PM}(2 \times n, \mathbb{R})$. Now, from item 1 in Lemma 3 with $n = 1$, we get

$$\begin{aligned} A^{-1} B H^{-1} &= A^{-1} B (A - B)^{-1} A (A + B)^{-1} \\ &= A^{-1} B (A + B)^{-1} A (A - B)^{-1} \\ &= A^{-1} B (I + A^{-1} B)^{-1} (A - B)^{-1} \\ &= (I - (I + A^{-1} B)^{-1}) (A - B)^{-1} = G_2 \end{aligned}$$

where we also used $(A - B)^{-1} A (A + B)^{-1} = (A + B)^{-1} A (A - B)^{-1}$ derived from item 2 in Lemma 3. Hence

$$\mathbf{N}_2^{-1} = I_2 \otimes (H^{-1} + H^{-1} B A^{-1}) - \mathbb{1}_2 \mathbb{1}_2^\top \otimes (H^{-1} B A^{-1}).$$

Assume that the claim holds for $r = p$. To extend the result to $r = p + 1$, again by [63, Block matrix inverse formula (0.8.5.6)] and [63, Woodbury inversion formula (0.7.4.1)], we have

$$\begin{aligned} \mathbf{N}_{p+1}^{-1} &= \begin{bmatrix} A & L_p^\top \\ L_p & \mathbf{N}_p \end{bmatrix}^{-1} \\ &= \begin{bmatrix} P^{-1} & -P^{-1} L_p^\top \mathbf{N}_p^{-1} \\ -\mathbf{N}_p^{-1} L_p P^{-1} & (\mathbf{N}_p - L_p A^{-1} L_p)^{-1} \end{bmatrix} \end{aligned}$$

where $P = A - L_p^\top \mathbf{N}_p^{-1} L_p$ and $L_p = \mathbb{1}_p \otimes B$. Let $\mathbf{N}_p^{-1} = I_p \otimes (F_p + G_p) - \mathbb{1}_p \mathbb{1}_p^\top \otimes G_p$, where

$$\begin{aligned} F_p &= (A - (p - 1)B (A + (p - 2)B)^{-1} B)^{-1} \\ G_p &= (A + (p - 1)B)^{-1} B (A - B)^{-1} \end{aligned}$$

where the inversions are valid from item 2 of Lemma 2. By simplification, we get $P = A - pB(F_p - (p - 1)G_p)B$,

and from items 1 and 2 of Lemma 3 for $n = p - 1$, we have

$$\begin{aligned} F_p - (p - 1)G_p \\ &= (A - (p - 1)B (A + (p - 2)B)^{-1} B)^{-1} \\ &\quad - (p - 1) (A + (p - 1)B)^{-1} B (A - B)^{-1} \\ &= (A + (p - 1)B)^{-1} (A + (p - 2)B) (A - B)^{-1} \\ &\quad - (p - 1) (A + (p - 1)B)^{-1} B (A - B)^{-1} \\ &= (A + (p - 1)B)^{-1} \end{aligned}$$

where the first term on the right-hand side of the first equation undergoes items 1 and 2 in Lemma 3, consecutively. And, the last equality follows by a direct simplification. The latter established equality results in $P = A - pB(A + (p - 1)B)^{-1}B$. Next, considering the off-diagonal blocks of $\mathbf{N}_{p+1}^{-1}$, with some simplification, each block of $P^{-1} L_p^\top \mathbf{N}_p^{-1}$ is equivalent to $P^{-1} B(F_p - (p - 1)G_p)$, and using the previous reasoning and Lemma 3, it can be simplified to

$$\begin{aligned} P^{-1} B (F_p - (p - 1)G_p) \\ &= P^{-1} B (A + (p - 1)B)^{-1} \\ &= (A + pB)^{-1} (A + (p - 1)B) \\ &\quad (A - B)^{-1} B (A + (p - 1)B)^{-1} \\ &= (A + pB)^{-1} B (A - B)^{-1}. \end{aligned}$$

Similarly, each block of $\mathbf{N}_p^{-1} L_p P^{-1}$ is also equal to $(A + (p - 1)B)^{-1} B (A - B)^{-1}$. Therefore, it only remains to show that the blocks of $(\mathbf{N}_p - L_p A^{-1} L_p)^{-1}$ are consistent with the desired pattern in $\mathbf{N}_{p+1}^{-1}$. Note that $\mathbf{N}_p - L_p A^{-1} L_p = I \otimes (A - B) + \mathbb{1} \mathbb{1}^\top \otimes (B - B A^{-1} B)$. Hence, with some algebraic rigor, we can show that each diagonal term of $(\mathbf{N}_p - L_p A^{-1} L_p)^{-1}$ is $(A + pB)^{-1} (A + (p - 1)B) (A - B)^{-1}$, and each off-diagonal becomes $-(A + pB)B(A - B)^{-1}$. Hence

$$\mathbf{N}_{p+1}^{-1} = I \otimes (F_{p+1} + G_{p+1}) - \mathbb{1} \mathbb{1}^\top \otimes G_{p+1}$$

with F_{p+1} and G_{p+1} defined as

$$\begin{aligned} F_{p+1} &= (A + pB)^{-1} (A + (p - 1)B) (A - B)^{-1} \\ G_{p+1} &= (A + pB) B (A - B)^{-1}. \end{aligned}$$

- 4) With direct multiplication and using the mixed-product property of Kronecker products, we have

$$\begin{aligned} \mathbf{N}_r \mathbf{M}_r &= I_r \otimes (A - B) (C - D) + \mathbb{1}_r \mathbb{1}_r^\top \otimes B (C - D) \\ &\quad + \mathbb{1}_r \mathbb{1}_r^\top \otimes (A - B) D + r \mathbb{1}_r \mathbb{1}_r^\top \otimes B D \\ &= I_r \otimes (A - B) (C - D) \\ &\quad + \mathbb{1}_r \mathbb{1}_r^\top \otimes (B (C - D) \\ &\quad + (A - B) D + r B D). \end{aligned} \quad \square$$

REFERENCES

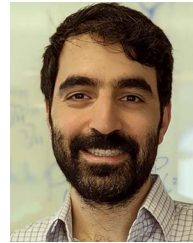
- [1] S. Talebi, S. Alemzadeh, and M. Mesbahi, "Distributed model-free policy iteration for networks of homogeneous systems," in *Proc. IEEE 60th Conf. Decis. Control*, pp. 6970–6975, 2021.
- [2] D. A. Bini, B. Iannazzo, and B. Meini, *Numerical Solution of Algebraic Riccati Equations*. Philadelphia, PA, USA: Soc. Ind. Appl. Math., 2011.
- [3] E. Camponogara, D. Jia, B. H. Krogh, and S. Talukdar, "Distributed model predictive control," *IEEE Control Syst. Mag.*, vol. 22, no. 1, pp. 44–52, Feb. 2002.
- [4] C. H. Papadimitriou and J. Tsitsiklis, "Intractable problems in control theory," *SIAM J. Control Optim.*, vol. 24, no. 4, pp. 639–654, 1986.
- [5] D. M. Stipanović, G. Inalhan, R. Teo, and C. J. Tomlin, "Decentralized overlapping control of a formation of unmanned aerial vehicles," *Automatica*, vol. 40, no. 8, pp. 1285–1296, 2004.
- [6] F. Borrelli, T. Keviczky, G. J. Balas, G. Stewart, K. Fregene, and D. Godbole, "Hybrid decentralized control of large scale systems," in *Proc. Int. Workshop Hybrid Syst., Comput. Control*, 2005, pp. 168–183.
- [7] F. Borrelli and T. Keviczky, "Distributed LQR design for identical dynamically decoupled systems," *IEEE Trans. Autom. Control*, vol. 53, no. 8, pp. 1901–1912, Sep. 2008.
- [8] P. Deshpande, P. P. Menon, C. Edwards, and I. Postlethwaite, "A distributed control law with guaranteed LQR cost for identical dynamically coupled linear systems," in *Proc. Amer. Control Conf.*, 2011, pp. 5342–5347.
- [9] W. Wang, F. Zhang, and C. Han, "Distributed linear quadratic regulator control for discrete-time multi-agent systems," *IET Control Theory Appl.*, vol. 11, no. 14, pp. 2279–2287, 2017.
- [10] P. Massioni and M. Verhaegen, "Distributed control for identical dynamically coupled systems: A decomposition approach," *IEEE Trans. Autom. Control*, vol. 54, no. 1, pp. 124–135, Jan. 2009.
- [11] J. Engwerda, *LQ Dynamic Optimization and Differential Games*. Hoboken, NJ, USA: Wiley, 2005.
- [12] V. Blondel and J. N. Tsitsiklis, "NP-hardness of some linear control design problems," *SIAM J. Control Optim.*, vol. 35, no. 6, pp. 2118–2127, 1997.
- [13] V. Gupta, B. Hassibi, and R. M. Murray, "A sub-optimal algorithm to synthesize control laws for a network of dynamic agents," *Int. J. Control*, vol. 78, no. 16, pp. 1302–1313, 2005.
- [14] M. Rotkowitz and S. Lall, "A characterization of convex problems in decentralized control," *IEEE Trans. Autom. Control*, vol. 50, no. 12, pp. 1984–1996, Feb. 2006.
- [15] B. Bamieh, F. Paganini, and M. A. Dahleh, "Distributed control of spatially invariant systems," *IEEE Trans. Autom. Control*, vol. 47, no. 7, pp. 1091–1107, Jul. 2002.
- [16] K. Mårtensson and A. Rantzer, "Gradient methods for iterative distributed control synthesis," in *Proc. IEEE 48th Conf. Decis. Control/28th Chin. Control Conf.*, 2009, pp. 549–554.
- [17] J. Bu, A. Mesbahi, M. Fazel, and M. Mesbahi, "LQR through the lens of first order methods: Discrete-time case," 2019, *arXiv:1907.08921*.
- [18] J. Bu, A. Mesbahi, and M. Mesbahi, "On topological properties of the set of stabilizing feedback gain," *IEEE Trans. Autom. Control*, vol. 66, no. 2, pp. 730–744, Feb. 2021.
- [19] S. J. Bradtke, B. E. Ydstie, and A. G. Barto, "Adaptive linear quadratic control using policy iteration," in *Proc. Amer. Control Conf.*, 1994, pp. 3475–3475.
- [20] S. J. Bradtke, "Incremental dynamic programming for online adaptive optimal control," Ph.D. dissertation, Dept. Comput. Sci., Univ. Massachusetts Amherst, Amherst, MA, USA, 1994.
- [21] G. Hewer, "An iterative technique for the computation of the steady state gains for the discrete optimal regulator," *IEEE Trans. Autom. Control*, vol. AC-16, no. 4, pp. 382–384, Aug. 1971.
- [22] D. McFadden, "On the controllability of decentralized macroeconomic systems: The assignment problem," in *Mathematical Systems Theory and Economics III*. New York, NY, USA: Springer, 1969, pp. 221–239.
- [23] S. H. Wang and E. Davison, "On the stabilization of decentralized control systems," *IEEE Trans. Autom. Control*, vol. AC-18, no. 5, pp. 473–478, Oct. 1973.
- [24] N. Sandell, P. Varaiya, M. Athans, and M. Safonov, "Survey of decentralized control methods for large scale systems," *IEEE Trans. Autom. Control*, vol. AC-23, no. 2, pp. 108–128, Apr. 1978.
- [25] P. Ioannou, "Decentralized adaptive control of interconnected systems," *IEEE Trans. Autom. Control*, vol. AC-31, no. 4, pp. 291–298, Apr. 1986.
- [26] J. A. Fax and R. M. Murray, "Information flow and cooperative control of vehicle formations," *IEEE Trans. Autom. Control*, vol. 49, no. 9, pp. 1465–1476, Sep. 2004.
- [27] P. Deshpande, P. Menon, C. Edwards, and I. Postlethwaite, "Sub-optimal distributed control law with H_2 performance for identical dynamically coupled linear systems," *IET Control Theory Appl.*, vol. 6, no. 16, pp. 2509–2517, 2012.
- [28] N. Motee and A. Jadbabaie, "Optimal control of spatially distributed systems," *IEEE Trans. Autom. Control*, vol. 53, no. 7, pp. 1616–1629, Aug. 2008.
- [29] A. Chapman, A. D. González, M. Mesbahi, L. Dueñas-Osorio, and R. M. D'Souza, "Data-guided control: Clustering, graph products, and decentralized control," in *Proc. IEEE 56th Annu. Conf. Decis. Control*, 2017, pp. 493–498.
- [30] S. Alemzadeh and M. Mesbahi, "Influence models on layered uncertain networks: A guaranteed-cost design perspective," in *Proc. IEEE Conf. Decis. Control*, 2018, pp. 5251–5256.
- [31] Y. R. Stürz, A. Eichler, and R. S. Smith, "Distributed control design for heterogeneous interconnected systems," *IEEE Trans. Autom. Control*, vol. 66, no. 11, pp. 5112–5127, Nov. 2021.
- [32] S. Vasisht, "Data-guided estimation and tracking methods for unmanned aerial vehicles," Ph.D. dissertation, Dept. Aeronaut. Astronaut., Univ. Washington, Seattle, WA, USA, 2019.
- [33] P. Dorato, V. Cerone, and C. Abdallah, *Linear-Quadratic Control: An Introduction*. New York, NY, USA: Simon and Schuster, 1994.
- [34] P. P. Khargonekar, I. R. Petersen, and K. Zhou, "Robust stabilization of uncertain linear systems: Quadratic stabilizability and H^∞ control theory," *IEEE Trans. Autom. Control*, vol. 35, no. 3, pp. 356–361, Mar. 1990.
- [35] Z. Li, Z. Duan, L. Xie, and X. Liu, "Distributed robust control of linear multi-agent systems with parameter uncertainties," *Int. J. Control*, vol. 85, no. 8, pp. 1039–1050, 2012.
- [36] L. Ljung, *System Identification: Theory for the User*. London, U.K.: Pearson Education, 1998.
- [37] H. J. Van Waarde, J. Eising, H. L. Trentelman, and M. K. Camlibel, "Data informativity: A new perspective on data-driven analysis and control," *IEEE Trans. Autom. Control*, vol. 65, no. 11, pp. 4753–4768, Nov. 2020.
- [38] A. Alaeddini, S. Alemzadeh, A. Mesbahit, and M. Mesbahi, "Linear model regression on time-series data: Non-asymptotic error bounds and applications," in *Proc. IEEE Conf. Decis. Control*, 2018, pp. 2259–2264.
- [39] S. Dean, H. Mania, N. Matni, B. Recht, and S. Tu, "On the sample complexity of the linear quadratic regulator," *Found. Comput. Math.*, vol. 20, pp. 633–679, 2020.
- [40] S. Talebi, S. Alemzadeh, N. Rahimi, and M. Mesbahi, "Online regulation of unstable linear systems from a single trajectory," in *Proc. IEEE 59th Conf. Decis. Control*, 2020, pp. 4784–4789.
- [41] S. Oymak and N. Ozay, "Non-asymptotic identification of LTI systems from a single trajectory," in *Proc. Amer. Control Conf.*, 2019, pp. 5655–5661.
- [42] F. L. Lewis, D. Vrabie, and K. G. Vamvoudakis, "Reinforcement learning and feedback control: Using natural decision methods to design optimal adaptive controllers," *IEEE Control Syst. Mag.*, vol. 32, no. 6, pp. 76–105, Dec. 2012.
- [43] Y. Luo, Z. Yang, Z. Wang, and M. Kolar, "Natural actor-critic converges globally for hierarchical linear quadratic regulator," 2019, *arXiv:1912.06875*.
- [44] S. Alemzadeh and M. Mesbahi, "Distributed Q-learning for dynamically decoupled systems," in *Proc. Amer. Control Conf.*, 2019, pp. 772–777.
- [45] T.-J. Chang and S. Shahrampour, "Distributed online linear quadratic control for linear time-invariant systems," in *Proc. Amer. Control Conf.*, 2021, pp. 923–928.
- [46] J. Li, H. Modares, T. Chai, F. L. Lewis, and L. Xie, "Off-policy reinforcement learning for synchronization in multiagent graphical games," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 28, no. 10, pp. 2434–2445, Oct. 2017.
- [47] S. Talebi, S. Alemzadeh, L. J. Ratliff, and M. Mesbahi, "Distributed learning in network games: A dual averaging approach," in *Proc. IEEE 58th Conf. Decis. Control*, 2019, pp. 5544–5549.
- [48] A. Nowé, P. Vrancx, and Y.-M. De Hauwere, "Game theory and multi-agent reinforcement learning," in *Reinforcement Learning*. New York, NY, USA: Springer, 2012, pp. 441–470.
- [49] G. C. Goodwin and K. S. Sin, *Adaptive Filtering Prediction and Control*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1984.
- [50] J. C. Willems, P. Rapisarda, I. Markovsky, and B. L. De Moor, "A note on persistency of excitation," *Syst. Control Lett.*, vol. 54, no. 4, pp. 325–329, 2005.

- [51] Y. Yu, S. Talebi, H. J. van Waarde, U. Topcu, M. Mesbahi, and B. Açikmeşe, "On controllability and persistency of excitation in data-driven control: Extensions of Willems' fundamental lemma," in *Proc. Conf. Decis. Control*, 2021, pp. 6485–6490.
- [52] S. Talebi, S. Alemzadeh, N. Rahimi, and M. Mesbahi, "On regularizability and its application to online control of unstable LTI systems," *IEEE Trans. Autom. Control*, vol. 67, no. 12, pp. 6413–6428, Dec. 2022.
- [53] Y. Jiang and Z.-P. Jiang, "Computational adaptive optimal control for continuous-time linear systems with completely unknown dynamics," *Automatica*, vol. 48, no. 10, pp. 2699–2704, 2012.
- [54] F. L. Lewis and K. G. Vamvoudakis, "Reinforcement learning for partially observable dynamic processes: Adaptive dynamic programming using measured output data," *IEEE Trans. Syst., Man, Cybern. B, Cybern.*, vol. 41, no. 1, pp. 14–25, Feb. 2011.
- [55] P. Comon and G. H. Golub, "Tracking a few extreme singular values and vectors in signal processing," *Proc. IEEE*, vol. 78, no. 8, pp. 1327–1343, Aug. 1990.
- [56] S. Haykin, *Adaptive Filter Theory*. Hoboken, NJ, USA: Prentice-Hall, 2002.
- [57] X. Chen and E. Hazan, "Black-box control for linear dynamical systems," in *Proc. 34th Conf. Learn. Theory*, 2021, pp. 1114–1143.
- [58] Y. Abbasi-Yadkori and C. Szepesvári, "Regret bounds for the adaptive control of linear quadratic systems," in *Proc. 24th Annu. Conf. Learn. Theory*, 2011, pp. 1–26.
- [59] N. Agarwal, B. Bullins, E. Hazan, S. Kakade, and K. Singh, "Online control with adversarial disturbances," in *Proc. 36th Int. Conf. Mach. Learn.*, 2019, pp. 111–119.
- [60] Y. S. Hung and A. MacFarlane, *Multivariable Feedback: A Quasi-Classical Approach*. New York, NY, USA: Springer-Verlag, 1982.
- [61] R. Bellman, "Dynamic programming," *Science*, vol. 153, no. 3731, pp. 34–37, 1966.
- [62] W. Levine, T. Johnson, and M. Athans, "Optimal limited state variable feedback controllers for linear systems," *IEEE Trans. Autom. Control*, vol. AC-16, no. 6, pp. 785–793, Dec. 1971.
- [63] R. A. Horn and C. R. Johnson, *Matrix Analysis*. Cambridge, U.K.: Cambridge Univ. Press, 2012.
- [64] P. Lancaster, "Explicit solutions of linear matrix equations," *SIAM Rev.*, vol. 12, no. 4, pp. 544–566, 1970.
- [65] M. A. Woodbury, "Inverting modified matrices," *Statist. Res. Group Memorandum Rep.*, vol. 42, no. 106, p. 336, 1950.
- [66] B. Bollobás, *Modern Graph Theory*, vol. 184. New York, York, USA: Springer, 2013.
- [67] G. C. Goodwin, S. F. Graebe, and M. E. Salgado, *Control System Design*. Upper Saddle River, NJ, USA: Prentice-Hall, 2001.
- [68] T. Mori, N. Fukuma, and M. Kuwahara, "On the discrete Lyapunov matrix equation," *IEEE Trans. Autom. Control*, vol. AC-27, no. 2, pp. 463–464, Apr. 1982.



Siavash Alemzadeh received the B.S. degree in mechanical engineering from the Sharif University of Technology, Tehran, Iran, in 2014, and the M.S. degree in mechanical engineering and the Ph.D. degree in aeronautics and astronautics from the University of Washington, Seattle, WA, USA, in 2015 and 2020, respectively.

He is currently a Data and Applied Scientist with Microsoft Corporation, Redmond, WA, USA. His research interests include reinforcement learning, data-driven control of distributed systems, and networked dynamical systems and their applications in transportation, infrastructure networks, and robotics.



Shahriar Talebi (Member, IEEE) received the B.Sc. degree in electrical engineering from the Sharif University of Technology, Tehran, Iran, in 2014, and the M.Sc. degree in electrical engineering from the University of Central Florida, Orlando, FL, USA, in 2017, the M.Sc. degree in mathematics, focusing on differential geometry, in 2023, and the Ph.D. degree in aeronautics and astronautics, specializing in control theory from the University of Washington, Seattle, WA, USA, in 2023.

He is currently a Postdoctoral Fellow with the School of Engineering and Applied Sciences, Department of Statistics, Harvard University, Cambridge, MA, USA. His research interests include control theory, differential geometry, learning for control, networked dynamical systems, and game theory.

Dr. Talebi was the recipient of the 2022 Excellence in Teaching Award at the University of Washington (UW). He is also a recipient of a number of scholarships, including the William E. Boeing Endowed Fellowship, the Paul A. Carlstedt Endowment, and the Latvian Arctic Pilot-A. Vagners Memorial Scholarship (UW, 2018–2019), as well as the Frank Hubbard Engineering Scholarship (University of Central Florida, 2017).



Mehran Mesbahi (Fellow, IEEE) received the Ph.D. degree from the University of Southern California, Los Angeles, CA, USA, in 1996.

From 1996 to 2000, he was a Member of the Guidance, Navigation, and Analysis Group, Jet Propulsion Laboratory, Pasadena, CA. From 2000 to 2002, he was an Assistant Professor of Aerospace Engineering and Mechanics with the University of Minnesota, Minneapolis, MN, USA. He is currently a Professor of Aeronautics and Astronautics and an Adjunct Professor of Electrical and Computer Engineering and Mathematics with the University of Washington (UW), Seattle, WA, USA, where he is also the Executive Director of the Joint Center for Aerospace Technology Innovation and a Member of the Washington State Academy of Sciences. His research interests include distributed and networked aerospace systems, systems and control theory, and learning.

Dr. Mesbahi was the recipient of the National Science Foundation CAREER Award, the NASA Space Act Award, the UW Distinguished Teaching Award, and the UW College of Engineering Innovator Award for Teaching.